

ULTRA-DEEP RESEARCH ANALYSIS

DynNav

Dynamic Navigation & Rerouting in Unknown Environments

Tutorial · Research Monograph · Technical Report · Beginner Guide

Panagiota Grosdouli

Electrical & Computer Engineering · Democritus University of Thrace

github.com/PanagiotaGr/DynNav-Dynamic-Navigation-Rerouting-in-Unknown-Environments

26 Contributions · 72 Tests · ROS 2 Humble · TurtleBot3

How to Read This Document

This document analyses all 26 research contributions of the DynNav project with ten sections per contribution:

- ① Beginner explanation — no background required
- ② Intuition with everyday analogies
- ③ Exactly what was implemented, step by step
- ④ Full algorithm, pseudocode, and mathematics
- ⑤ Research question and novelty
- ⑥ Results and what they prove
- ⑦ Reviewer-level critique — weaknesses, assumptions, failure cases
- ⑧ How to turn this into a publishable paper
- ⑨ Open research questions this work unlocks
- ⑩ Two-minute plain-language summary

Every contribution also receives four scores (1–10): Originality, Technical Depth, Publishability, PhD Potential.

Part I — The Big Picture

What is DynNav in One Sentence?

A robot enters a building it has never seen. It must reach a goal safely. It has imperfect sensors, limited battery, potential adversaries, and no map. DynNav is the complete thinking-and-acting system that makes this possible.

Why This Problem Matters

Classical navigation assumes three things that are false in the real world: (1) the map is complete and correct, (2) sensors are perfect, (3) the environment is static. Every contribution in DynNav attacks one or more of these false assumptions.

The 26-Module Architecture

Sensing & Uncertainty	02 EKF/UKF · 15 DVS+SNN · 23 3D-GS · 24 NeRF
Risk-Aware Planning	01 Learned A* · 03 CVaR A* · 06 Energy/Conn · 07 NBV
Formal Safety	04 Returnability · 05 Safe-Mode · 18 STL+CBF Shields
Security & Diagnosis	08 IDS · 14 Causal SCM · 20 Failure Explainer · 25 Attacks
Foundation Models	11 VLM Goals · 17 Topo Maps · 19 LLM Planner
Generative AI	12 Diffusion Occupancy · 13 World Model (RSSM)
RL & Learning	21 PPO Agent · 22 Curriculum RL
Multi-Robot	09 Coordination · 16 Federated · 26 BFT Swarm
Human-Robot	10 Ethics/Trust

Part II — All 26 Contributions: Ultra-Deep Analysis

Contribution 01 Learned A* Heuristics

Category	Planning / Learned Heuristics
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Finding a path from A to B in a grid map is what A* does. Its speed depends on how well it guesses which direction the goal is in — the heuristic. A bad heuristic explores too many dead ends. This module trains a neural network to make better guesses from past experience.

Why does this problem exist?

A* was designed for small, simple maps. In large buildings with hundreds of rooms, exploring thousands of cells before finding a path is too slow for real-time replanning. A smarter heuristic means fewer cells explored.

Why is it hard?

Any heuristic that overestimates the true distance to the goal will break A*'s optimality guarantee. So you cannot just use any neural prediction — you must mathematically ensure it never overestimates, using a clipping operation.

② Intuition-First Explanation

A taxi driver who has driven the city for 20 years does not need GPS to know roughly which direction the airport is. Their internalized knowledge guides them faster than someone checking every possible turn. The learned heuristic gives the robot that same internalized spatial knowledge.

Analogy: An experienced chess player does not evaluate every possible move — they immediately dismiss most and focus on a few promising ones. The neural heuristic is the chess intuition that says 'the goal is roughly that way' — focusing the A search on the right part of the map.*

③ What I Built — Step by Step

- Collected (state, true_cost_to_goal) pairs by running optimal A* on many training environments
- State features: relative position to goal, local occupancy window, distance metrics
- Trained MLP regressor: minimize $MSE(h_{\theta}(s), h^*(s))$ over all collected pairs
- At inference: $\tilde{h}(s) = \min(h_{\theta}(s), \text{manhattan}(s, \text{goal}))$ — clipping ensures admissibility
- Ran standard A* with $\tilde{h}$ as heuristic — same algorithm, better guidance
- Eval script: `contributions/01_learned_astar/experiments/eval_astar_learned.py`

- Metrics: node expansions, path length ratio, runtime

Data Flow

Input: (robot_pos, goal_pos, local_occ) → MLP → $h_{\theta}(s)$ → min with Manhattan → $\tilde{h}(s)$ → A priority queue → optimal path*

④ Deep Technical Breakdown

Key Components

- State encoder: encodes relative position, local occupancy window (NxN grid), goal direction
- MLP regressor: 2 hidden layers with ReLU, output = scalar $h_{\theta}(s)$
- Admissibility clipper: $\tilde{h} = \min(h_{\theta}, h_{\text{naive}})$ — provably $\leq h^*(s)$
- Standard A*: unchanged except heuristic function replaced with $\tilde{h}$

Pseudocode / Algorithm

```

TRAINING:
  for env in training_maps:
    run A*(env) → collect {(s, h*(s))} pairs
  train MLP: minimize (1/N)  $\sum (h_{\theta}(s_i) - h^*(s_i))^2$ 

INFERENCE:
  h_clipped(s, goal):
    h_neural = MLP(encode(s, goal))
    h_safe = manhattan(s, goal)
    return min(h_neural, h_safe) # admissibility guaranteed

run A* using h_clipped as heuristic

```

Mathematical Formulation

Admissibility: $\tilde{h}(s) = \min(h_{\theta}(s), h_{\text{naive}}(s)) \leq h_{\text{naive}}(s) \leq h^*(s)$ ✓
 Loss: $L(\theta) = (1/N) \sum_{i=1}^N (h_{\theta}(s_i) - h^*(s_i))^2$
 A* expansion: $f(n) = g(n) + \tilde{h}(n) \rightarrow$ expand lowest- f node first
 Optimality: preserved iff $\tilde{h}(s) \leq h^*(s) \forall s$ (guaranteed by clipping)

Complexity & Assumptions

Time complexity: A* unchanged: $O(b^d)$. Training: $O(N \cdot |\text{MLP}|)$. Inference per node: $O(|\text{MLP}|) \approx O(1)$.

- Training and test environments share the same structural distribution (rooms, corridors)
- Manhattan distance is always $\leq$ true cost (valid for grid worlds without non-Euclidean geometry)
- MLP has sufficient capacity to approximate $h^*(s)$ from features

⑤ Research Meaning

Research Question

Can a neural heuristic trained on past navigation episodes reduce A node expansions by $\geq 30\%$ while provably maintaining path optimality across unseen environments of the same structural class?*

Novelty

The core idea (neural heuristic + admissibility clipping) exists in the literature, but its systematic evaluation in the DynNav mobile navigation context — with structured feature encoding and cross-map generalization analysis — is a valid incremental contribution.

Related Work

Yonetani et al. (2021) Neural A* — differentiable A*. Takahashi et al. — motion planning heuristics. Lykouris & Vassilvitskii (2018) — learning-augmented algorithms theory.

⑥ Results & Evidence

~35% reduction in node expansions on benchmark maps. Path optimality preserved in all tested cases. Runtime improvement correlates with map complexity.

Metrics Used

- Node expansions (count of cells added to closed list)
- Path length ratio: learned/optimal A* (should be 1.0)
- Wall-clock planning time
- Generalization gap: seen vs. unseen map types

What Is Missing

No comparison with published neural heuristic methods (Neural A*, Differentiable A*). No evaluation on robot benchmark datasets. No statistical significance tests. No generalization analysis across map types.

⑦ Reviewer Critique

Weaknesses

- Min-clipping for admissibility is a well-known trick — not a research contribution on its own
- 35% improvement needs statistical confidence intervals across many runs
- No baseline against RRT*, Dijkstra, or Differentiable A*
- Consistency (monotonicity) not guaranteed by clipping — may slow A* in some cases

Hidden Assumptions

- Training distribution = test distribution (no domain shift)
- Manhattan is always a valid lower bound (fails for non-Euclidean cost functions)
- MLP can be evaluated in real-time (not profiled on embedded hardware)

Failure Scenarios

- Environments structurally very different from training (e.g., open outdoor spaces vs. indoor mazes)
- Maps with long narrow passages that confuse the spatial features
- Dense obstacle fields where Manhattan distance is extremely loose

⑧ How to Turn This Into a Paper

Paper Title	Admissible Neural Heuristics for Learning-Augmented A* Navigation in Structured Environments
--------------------	--

Venue	ICRA Workshop on Learning for Planning / RA-L
-------	---

Missing Experiments

- Systematic comparison with Neural A* (Yonetani 2021) on same benchmark maps
- Evaluation on MatterPort3D and Gibson robot navigation datasets
- Online learning variant: update heuristic as robot explores unknown space
- Statistical testing: t-test on node expansions across 100+ map instances

Ablation Studies Needed

- With vs. without clipping — does clipping cost any planning quality?
- Raw features vs. structured encoding — does feature design matter?
- Transfer: train on one map type, test on another — how does performance degrade?
- MLP size sensitivity: 1 layer vs. 2 vs. 3 hidden layers

⑨ Open Research Questions

1. Can the heuristic be learned online as the robot maps unknown space (online meta-learning)?
2. Can consistency (not just admissibility) be enforced as a training regularizer?
3. How does the learned heuristic behave near topological bottlenecks (doorways, junctions)?
4. Can this be extended to 3D navigation or continuous state spaces?

⑩ Two-Minute Explanation for Anyone

A robot finding its way through a building is like solving a maze. The standard algorithm (A) explores many dead ends before finding the goal. I trained a small neural network to make better guesses about which direction the goal is, so the algorithm explores fewer wrong paths. The key constraint: the neural guess must never be too optimistic — otherwise the algorithm stops guaranteeing the shortest path. A simple clipping operation enforces this. Result: 35% fewer cells explored, same quality path.*

Evaluation Scores

Dimension	Score	Comment
Originality	5/10	Standard method
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	6/10	Supporting work

Contribution 02 Uncertainty Estimation (EKF & UKF)

Category	Perception / State Estimation
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

A robot's sensors are wrong. LiDAR returns noisy measurements. Wheels slip and drift. If the robot treats sensor readings as ground truth, it will think it is in the wrong place and walk into walls. The solution: maintain a probability distribution over possible positions, not a single point estimate.

Why does this problem exist?

All physical sensors have noise — this is unavoidable physics. Without explicitly modelling this noise, downstream modules (planner, safety shields) make overconfident decisions that fail in practice.

Why is it hard?

The state transition (how the robot moves) and measurement (what the sensor reads) are nonlinear functions. Linear Kalman filtering is exact but wrong for nonlinear systems. EKF linearizes via Jacobians; UKF uses sigma points — both are approximations with different failure modes.

② Intuition-First Explanation

A blindfolded person walking through a house uses sound and touch to estimate position. They are not certain where they are, but they maintain a range of possibilities and update that range with each new clue. The Kalman filter is the mathematically optimal way to do this when noise is Gaussian.

Analogy: A detective does not say 'the suspect was definitely at location X at 8pm'. They say 'most likely between locations X and Y, based on three imperfect witnesses'. The EKF gives the robot the same probabilistic reasoning ability for its own position.

③ What I Built — Step by Step

- Implemented EKF for unicycle model: state $s_t = [x, y, \theta]^T$
- Process model: unicycle kinematics (nonlinear) + Gaussian noise $w_t \sim N(0, Q)$
- Measurement model: LiDAR range/bearing + odometry + Gaussian noise $v_t \sim N(0, R)$
- EKF predict: linearize via Jacobian $F_t = \partial f / \partial s$, propagate covariance
- EKF update: compute Kalman gain K_t , update mean and covariance
- UKF alternative: sigma-point propagation (no Jacobian needed)
- Output: belief (μ_t, Σ_t) fed to risk planner (C03) and IDS (C08)

Data Flow

Odometry control + LiDAR readings → EKF predict step → EKF update step → belief (μ_t, Σ_t) → risk planner / safety shield / IDS

④ Deep Technical Breakdown

Key Components

- State vector: $[x, y, \theta]^T$ — 2D position and heading
- EKF predict: propagate mean through nonlinear f , covariance through linearized F_t
- EKF update: Kalman gain K_t balances prediction vs. measurement trust
- UKF sigma points: $2n+1$ deterministic samples that capture mean and covariance
- Uncertainty scalar: $\text{tr}(\Sigma_t)$ — consumed by risk planner as risk amplifier

Pseudocode / Algorithm

```
EKF PREDICT:
     $\hat{\mu} = f(\mu_{t-1}, a_t)$  // propagate mean through motion model
     $\hat{P} = F_t \cdot P_{t-1} \cdot F_t^T + Q$  // propagate covariance, add process noise

EKF UPDATE:
     $K = \hat{P} \cdot H^T \cdot (H \cdot \hat{P} \cdot H^T + R)^{-1}$  // Kalman gain
     $\mu_t = \hat{\mu} + K \cdot (z_t - h(\hat{\mu}))$  // correct mean with measurement
     $P_t = (I - K \cdot H) \cdot \hat{P}$  // reduce uncertainty

UKF SIGMA POINTS (alternative):
     $\chi_0 = \mu$ ,  $\chi_{\{i\}} = \mu + \sqrt{(n+\lambda) \cdot P}_i$  for  $i=1..n$ 
     $\chi_{\{n+i\}} = \mu - \sqrt{(n+\lambda) \cdot P}_i$  for  $i=1..n$ 
    propagate all  $2n+1$  points through nonlinear  $f$ , compute weighted mean/cov
```

Mathematical Formulation

```
State-space:  $s_{t+1} = f(s_t, a_t) + w_t$ ,  $z_t = h(s_t) + v_t$ 
Noise:  $w_t \sim N(0, Q)$ ,  $v_t \sim N(0, R)$ 
Jacobians:  $F_t = \partial f / \partial s |_{\{\mu_{t-1}\}}$ ,  $H_t = \partial h / \partial s |_{\{\mu_t\}}$ 
Innovation:  $v_k = z_k - h(\mu_k) \sim N(0, S_k)$  under no attack
NEES:  $(s_t - \mu_t)^T P_t^{-1} (s_t - \mu_t) \sim \chi^2(n)$  if filter consistent
```

Complexity & Assumptions

Time complexity: $O(n^2)$ per step for EKF ($n=3$: negligible). $O(n^3)$ for UKF (matrix sqrt). Both real-time for $n \leq 10$.

- Gaussian noise — EKF/UKF fail if noise is bimodal or heavy-tailed
- Known Q and R — in practice these must be tuned manually
- No loop closure — EKF diverges in long corridors without external correction
- Single-hypothesis belief — cannot represent multimodal distributions (robot near symmetric junction)

⑤ Research Meaning

Research Question

How does the quality of belief-state uncertainty estimation affect downstream risk-aware planning and formal safety verification — and what is the minimum sensor quality needed for formal safety guarantees to hold?

Novelty

EKF/UKF are textbook methods (Thrun 2005). The contribution is systems integration: uncertainty as a first-class signal flowing to planning, safety, and security modules simultaneously. Not algorithmic novelty — infrastructure novelty.

Related Work

Thrun, Burgard, Fox — Probabilistic Robotics (2005). Wan & Merwe (2000) UKF. Montemerlo et al. (2002) FastSLAM. Julier & Uhlmann (1997) UKF original.

⑥Results & Evidence

Correct belief propagation validated in simulation. RMSE of position estimate measured vs. ground truth. Covariance trace used successfully as uncertainty scalar by risk planner.

Metrics Used

- RMSE of position estimate (m)
- Covariance trace $\text{tr}(\Sigma_t)$ over time
- NEES score — filter consistency
- Innovation bound violations (fed to IDS C08)

What Is Missing

No comparison EKF vs UKF on high-curvature trajectories. No real TurtleBot3 validation with actual IMU + LiDAR fusion. No long-duration divergence analysis. Q and R tuning not documented.

⑦Reviewer Critique

Weaknesses

- EKF/UKF are 50-year-old methods — no algorithmic contribution
- Gaussian assumption fails near symmetric junctions (particle filter needed)
- Manual Q,R tuning undocumented — reproducibility concern
- No loop closure → EKF diverges over long sessions

Hidden Assumptions

- Unimodal position distribution assumed
- Noise is stationary (false during acceleration)
- Process model f is exactly known (wheel diameter changes with wear)

Failure Scenarios

- Long corridors without landmarks → EKF diverges
- Symmetric environments → multimodal belief, EKF collapses to wrong mode
- Sensor dropout → pure prediction diverges
- Under adversarial attack (C08) → EKF incorporates corrupted measurements

⑧How to Turn This Into a Paper

Paper Title	Belief-State Uncertainty Infrastructure for Risk-Aware and Formally-Safe Robot Navigation
--------------------	---

Venue	Not standalone — Section 3 of a system paper (ICRA/RA-L)
-------	--

Missing Experiments

- EKF vs. UKF vs. particle filter on high-curvature trajectories in simulation
- Real TurtleBot3: fuse wheel odometry + RPLiDAR, validate against OptiTrack ground truth
- Long-duration sessions: 30+ minutes, measure filter consistency via NEES
- Q/R sensitivity: how does miscalibration affect downstream planning quality?

Ablation Studies Needed

- EKF vs. UKF: when does the sigma-point advantage matter?
- Effect of uncertainty quality on CVaR planning (C03) — does better filter → safer paths?
- EKF vs. particle filter in symmetric environment
- With vs. without uncertainty signal in cost function — does it improve safety?

⑨ Open Research Questions

5. Can Q and R be automatically calibrated from data without ground truth (EM algorithm)?
6. Can the covariance Σ_t be directly used as a CBF safety constraint (C18)?
7. What is the minimum sensor quality (R bound) for formal CBF safety guarantees to hold?
8. Can a differentiable EKF enable end-to-end uncertainty-aware planning?

⑩ Two-Minute Explanation for Anyone

A robot's sensors lie — not completely, but enough to matter. If the robot thinks it's at position A when it's actually at B, it might walk into a wall. The Kalman filter is the mathematical recipe for combining what sensors report with what the motion model predicts, weighted by how reliable each is. The result is a probability distribution over the robot's possible position — not a single point. This uncertainty estimate is then used by every other module: the risk planner, the safety shield, and the anomaly detector all know how uncertain the robot is about its location.

Evaluation Scores

Dimension	Score	Comment
Originality	4/10	Standard method
Technical Depth	7/10	Prototype level
Publishability	3/10	Concept only
PhD Potential	5/10	Supporting work

Contribution 03 Belief-Space Risk-Aware Planning (CVaR A*)

Category	Risk-Sensitive Planning
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

The shortest path is not always the safest. A shortcut that passes 2cm from a sharp corner is risky: any sensor error causes a collision. This module modifies the path planner to balance distance against risk, specifically using CVaR — a financial risk measure that focuses on worst-case scenarios.

Why does this problem exist?

Classical planners optimize only distance. In environments with sensor noise and uncertain maps, a path that is geometrically short but passes through high-uncertainty regions is practically dangerous. Risk-awareness is essential for deployment in real environments.

Why is it hard?

Formalizing 'risk' in a way the planner can optimize is non-trivial. Mean collision probability ignores rare but catastrophic events. CVaR solves this but requires sampling or closed-form computation from the belief state. Balancing risk and efficiency requires a tunable parameter with no universal optimal value.

② Intuition-First Explanation

A hiker chooses between a 5km mountain trail on a cliff edge (fast, dangerous) and a 7km valley route (slower, safe). Even a 15% chance of falling makes the cliff unacceptable. CVaR formalizes this: it focuses planning on the worst 5% of scenarios, not just the average.

Analogy: Insurance companies price coverage for tail risk — rare but catastrophic events — not for the average claim. CVaR is the actuarial tool for this. Applied to navigation: don't plan for the average obstacle configuration; plan for the worst plausible one.

③ What I Built — Step by Step

- Modified A* cost function: $f(s) = g(s) + \lambda \cdot \text{CVaR}(p_{\text{col}}(s)) + h(s)$
- Risk maps from: EKF uncertainty (C02) or diffusion occupancy samples (C12)
- CVaR computed as empirical mean of top $(1-\alpha) \cdot N$ worst samples
- Parameter λ swept systematically to generate safety-efficiency Pareto curve
- Ablation: risk-agnostic A* vs. risk-aware A* on 30+ simulated environments
- Eval script: `contributions/03_belief_risk_planning/experiments/eval_belief_risk.py`

Data Flow

Belief state (μ, Σ) + N occupancy samples $\rightarrow$ CVaR per cell $\rightarrow$ risk cost map $\rightarrow$ modified A $\rightarrow$ safe optimal path*

④ Deep Technical Breakdown

Key Components

- Risk map builder: per-cell CVaR from belief state or diffusion samples
- CVaR estimator: sort N samples, average top $(1-\alpha) \cdot N$ fraction
- Modified A*: standard algorithm with risk-augmented edge costs
- Pareto analyzer: sweeps $\lambda \in [0, \infty)$, records (path_length, risk_along_path)
- Plug-in interface: any probabilistic occupancy source $\rightarrow$ same planner

Pseudocode / Algorithm

```
RISK MAP CONSTRUCTION:
for each cell c in grid:
    samples = [occupancy_sample(c) for _ in range(N)]
    sorted_s = sort(samples)
    CVaR[c] = mean(sorted_s[ceil(alpha*N):])    // top (1-alpha) worst samples

RISK-AUGMENTED A*:
f(n) = g(n) + lambda * CVaR[n] + h(n)
open_list ordered by f(n)  -> standard A* expansion

PARETO CURVE:
for lambda in [0, 0.1, 0.5, 1.0, 2.0, 5.0, infinity]:
    path = risk_aware_Astar(map, start, goal, lambda)
    record (length(path), mean_CVaR_along_path)
```

Mathematical Formulation

```
CVaR_alpha(X) = E[X | X >= VaR_alpha(X)] = (1/(1-alpha)) * integral_alpha^1 VaR_u(X) du
Empirical: CVaR_hat_alpha = (1/|T|) * sum_{i in T} x_(i)   where T = top (1-alpha) * N indices
Augmented cost: f(s) = g(s) + lambda * CVaR_alpha(p_col(s)) + h(s)
Planning: pi* = argmin_{pi: s -> g} sum_{s in pi} [c(s) + lambda * CVaR_alpha(p_col(s))]
lambda=0: pure A*   |   lambda -> infinity: maximum risk aversion (stays at start in extreme case)
```

Complexity & Assumptions

Time complexity: Identical to A*: $O(b^d)$. CVaR computation: $O(N \log N)$ per cell, done once. Total overhead is in risk map construction, not planning.

- Risk = collision probability from static occupancy (ignores dynamic obstacles)
- CVaR sample set representative of true occupancy distribution
- Same λ optimal across all environments (in practice, λ is environment-specific)
- Per-cell risks are independent (ignores spatial correlations — actually they are strongly correlated)

⑤ Research Meaning

Research Question

What is the fundamental Pareto tradeoff between path efficiency and CVaR tail-risk in probabilistic robot navigation, and how does this tradeoff depend on environment structure and belief quality?

Novelty

The modular design — any probabilistic occupancy estimator (EKF, diffusion, NeRF) plugs into the same risk-augmented planner — is the key architectural contribution. CVaR planning for robotics is established (Majumdar & Pavone 2020) but this plug-in interface is novel.

Related Work

Majumdar & Pavone (2020) — CVaR motion planning. Ono & Williams (2008) — chance-constrained planning. Van den Berg et al. (2011) — LQG-MP. Herbert et al. (2017) — FaSTrack.

⑥Results & Evidence

Clear Pareto curve: increasing λ increases path length, decreases collision risk. Risk-aware planner avoids narrow corridors even when they are geometrically shorter. Qualitative demonstrations in simulation.

Metrics Used

- Path length (m)
- CVaR of collision probability along path
- Cells with occupancy > 0.5 within 1 robot radius
- Pareto curve area (efficiency summary)

What Is Missing

No comparison with chance-constrained or robust MPC planners. No dynamic environment evaluation. No hardware validation. λ selection criterion not provided.

⑦Reviewer Critique

Weaknesses

- CVaR planning for robotics is well-established — needs stronger differentiation
- Fixed λ is a limitation: no principled selection method
- Static risk map: does not adapt to moving obstacles
- Per-cell CVaR ignores spatial correlations

Hidden Assumptions

- Risk = occupancy probability (misses dynamic and intentional hazards)
- λ transfers across environments (unlikely)
- CVaR sample size N is sufficient (variance of estimate not analyzed)

Failure Scenarios

- High λ : robot refuses to move if all paths pass through risky cells
- Dynamic obstacles: risk map computed before movement becomes stale
- Overconfident belief: EKF underestimates $\sigma \rightarrow$ CVaR underestimates true risk

⑧How to Turn This Into a Paper

Paper Title	Modular CVaR-Augmented A* for Risk-Sensitive Navigation Under Probabilistic Occupancy Uncertainty
Venue	IROS / RA-L

Missing Experiments

- Comparison: CVaR A* vs. chance-constrained A* vs. risk-neutral A* on identical maps
- Dynamic environments: update risk map mid-execution, measure collision reduction
- Hardware: TurtleBot3 in a known-hazard environment
- λ selection study: which λ minimizes total cost (path length + collisions)?

Ablation Studies Needed

- CVaR_ α sensitivity: $\alpha=0.95$ vs. 0.8 vs. 0.5 — when does α matter?
- Static vs. dynamically updated risk map
- EKF-derived vs. diffusion-derived risk map — which produces better planning?
- Risk map resolution: coarse vs. fine grid

⑨ Open Research Questions

9. Can λ be learned from human preferences (inverse reward learning)?
10. Can CVaR be computed in closed form for Gaussian occupancy beliefs?
11. How does risk compound along a path — are adjacent cells independent or correlated?
12. Can risk-aware planning be embedded directly in RL reward shaping (C21)?

⑩ Two-Minute Explanation for Anyone

The shortest route isn't always best. A path that skims a dangerous corner might be 5 meters shorter but risks collision every time the sensor misfires. This module teaches the planner to weigh risk against distance using CVaR — a concept from finance that focuses on the worst-case 5% of scenarios, not the average. A single parameter λ controls the tradeoff: high λ means the robot takes longer routes to stay safe. The key design: any risk source (sensor uncertainty, diffusion maps, neural predictions) can plug into this same framework without changing the planner.

Evaluation Scores

Dimension	Score	Comment
Originality	6/10	Incremental
Technical Depth	8/10	Full maths + code
Publishability	7/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 04 Irreversibility & Returnability Constraints

Category	Safety / Dead-End Avoidance
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Some robot actions cannot be undone. Entering a dead-end corridor in an unknown building may leave the robot permanently stuck. This module checks, before every action, whether the robot will still be able to return to safety from the proposed next position.

Why does this problem exist?

Reactive planners commit to actions without considering reversibility. In unknown environments, a locally good move (take this corridor) may be globally catastrophic (it leads to a dead end). Proactive returnability checking prevents this class of failures.

Why is it hard?

Checking returnability requires solving a backward reachability problem — computationally expensive. BFS approximation is tractable but sacrifices formal guarantees. The horizon parameter H must be set correctly or the check gives false confidence.

② Intuition-First Explanation

Before descending into an unfamiliar mountain ravine, a cautious hiker asks: 'If the weather turns, can I climb back up?' The robot asks the same question before entering any cell: 'If the map turns out worse than I think, can I still reach safety?' This one question prevents a class of catastrophic failures.

Analogy: A chess player avoids moves that 'play themselves into a corner' — not just avoiding immediate loss, but ensuring there is always an escape route several moves ahead. Returnability is the chess concept of maintaining maneuverability, formalized for robot navigation.

③ What I Built — Step by Step

- Defined safe set S_{safe} : designated base position (charging station, entrance)
- Backward BFS from S_{safe} on current occupancy grid
- Computed $d_G(s, S_{\text{safe}})$ for all reachable cells in $O(W \cdot H)$
- Returnability: $\text{Ret}_H(s) = 1$ if $d_G(s, S_{\text{safe}}) \leq H$ else 0
- Modified A* cost: $f(s) = g(s) + \lambda \cdot \text{CVaR}(s) + \gamma \cdot (1 - \text{Ret}_H(s)) + h(s)$
- Two modes: soft (penalty γ) and hard (prune non-returnable from open list)
- Eval: `contributions/04_irreversibility_returnability/experiments/eval_returnability.py`

Data Flow

Current map + S_safe → backward BFS → $d_G(s, S_safe) \forall s \rightarrow Ret_H(s) \rightarrow$ modified A cost → returnability-safe path*

④ Deep Technical Breakdown

Key Components

- Backward BFS: $O(W \cdot H)$ sweep from S_safe , labels all reachable distances
- Returnability predicate: $Ret_H(s) = 1[d_G(s, S_safe) \leq H]$
- Cost augmentation: $c_irrev(s) = \gamma \cdot (1 - Ret_H(s))$ added to A* edge cost
- Hard-constraint mode: remove non-returnable cells from A* consideration

Pseudocode / Algorithm

```
BACKWARD BFS FROM S_safe:
  dist = {s: ∞ for all s}
  dist[S_safe] = 0; queue = deque([S_safe])
  while queue:
    s = queue.popleft()
    for neighbor n of s:
      if occ[n] < 0.5 and dist[n] == ∞:
        dist[n] = dist[s] + 1
        queue.append(n)

RETURNABILITY CHECK:
  Ret_H(s) = 1 if dist[s] <= H else 0

MODIFIED A* (soft mode):
  f(n) = g(n) + λ*CVaR(n) + γ*(1-Ret_H(n)) + h(n)
```

Mathematical Formulation

```
Returnability:    Ret_H(s) = 1[d_G(s, S_safe) ≤ H]
d_G: graph shortest path on current occupancy grid
Irrev. penalty:   c_irrev(s) = γ · (1 - Ret_H(s)) ∈ {0, γ}
BRT(S_safe):     {s : ∃ path from s to S_safe on current map} ← true backward
reachable set
BFS approximation: BRT ∩ {s : d_G(s, S_safe) ≤ H} — conservative (may miss distant
returnable states)
```

Complexity & Assumptions

Time complexity: BFS precomputation: $O(W \cdot H)$. Per-action lookup: $O(1)$. Memory: $O(W \cdot H)$ for distance map.

- Safe set S_safe is static and initially reachable
- BFS horizon H is sufficient to capture all relevant returnability constraints
- Current occupancy map is accurate enough to assess returnability
- Robot can reverse (non-holonomic constraints not considered)

⑤ Research Meaning

Research Question

Can preemptive BFS-horizon returnability checking provably reduce the rate of irreversible state entry in unknown environments, compared to a planner that optimizes only distance and risk?

Novelty

Connecting navigation to safe exploration theory (Moldovan & Abbeel 2012) via a practical BFS-horizon approximation. The integration with CVaR planning (C03) and world-model pre-screening (C13) creates a multi-layer irreversibility defense.

Related Work

Moldovan & Abbeel (2012) — safe exploration in MDPs. Gillula & Tomlin (2011) — HJ reachability. Schmerling et al. — backward reachability for motion planning.

⑥ Results & Evidence

Qualitative demonstration: robot avoids dead-end corridors. Rate of irreversible state entry reduced vs. baseline A*. Path length overhead measured.

Metrics Used

- Irreversible state entries per episode
- Mission completion rate
- Path length overhead of returnability constraint
- Horizon H sensitivity

What Is Missing

No formal proof that BFS horizon $\approx$ backward reachability. No comparison with exact HJ reachability. No analysis of BFS error rate under uncertain maps. No hardware validation.

⑦ Reviewer Critique

Weaknesses

- BFS horizon is an approximation — formal backward reachability not computed
- H must be tuned — no principled selection method
- Does not account for non-holonomic dynamics (car-like robots cannot reverse freely)
- Under uncertain maps, returnable states may be incorrectly flagged

Hidden Assumptions

- S_{safe} is always reachable at episode start
- Robot dynamics allow reversal (not always true)
- Occupancy map is accurate for BFS (may be wrong in unexplored areas)

Failure Scenarios

- Dynamic obstacles block the return path after the robot passes
- H too small: BFS misses returnable states, overly conservative
- Near S_{safe}: returnability trivially satisfied, no safety added
- Non-holonomic robot: BFS path may not be kinematically feasible

⑧ How to Turn This Into a Paper

Paper Title	Returnability-Constrained Navigation: Safe Exploration via Backward Reachability Approximation in Unknown Environments
--------------------	--

Venue	ICRA Safety Workshop / RA-L
-------	-----------------------------

Missing Experiments

- Comparison with exact HJ reachability on identical scenarios
- Monte Carlo: how often does BFS-H incorrectly classify returnable states?
- Real robot: TurtleBot3 in maze — does it ever get permanently stuck?
- H sensitivity: sweep H and measure irreversible entry rate

Ablation Studies Needed

- Soft vs. hard returnability constraint — which is more effective?
- H sensitivity on different environment topologies
- With vs. without occupancy uncertainty inflation in BFS
- Returnability alone vs. returnability + CVaR — do they combine well?

⑨ Open Research Questions

13. Can probabilistic returnability ($P(\text{returnable}) \geq p$) be computed tractably?
14. Can returnability be combined with CBF (C18) for continuous-time formal guarantees?
15. How should S_{safe} be chosen dynamically as the robot explores?
16. Can returnability be learned from demonstrations (imitation of cautious behavior)?

⑩ Two-Minute Explanation for Anyone

Before taking any action, the robot asks: 'If I go there, can I get back to safety?' It checks this by running a quick backward search from the safe zone on its current map, flagging any cell that's too far from safety. Cells that fail the check are avoided or penalized in the path planning cost. This prevents the robot from ever walking into a dead end it cannot escape. Think of it as the robot's version of a mountain hiker's rule: never go somewhere you cannot get back from.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 05 Safe-Mode Navigation FSM

Category	Safety / Adaptive Behavior
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

A robot navigating normally (fast, aggressive) through a low-risk corridor should behave very differently near a crowded hospital waiting room. A single fixed navigation policy cannot handle both cases. This module implements automatic behavioral mode switching based on real-time risk level.

Why does this problem exist?

Fixed-parameter navigation is either too conservative (wastes time when safe) or too aggressive (dangerous in high-risk situations). Adaptive behavior is fundamental to safe deployment across diverse environments.

Why is it hard?

The switching logic must avoid rapid oscillation (hysteresis needed) without being too slow to respond to sudden danger. Six hyperparameters must be tuned. The risk signal quality determines the value of the whole module.

② Intuition-First Explanation

A car driver naturally slows down in fog, school zones, and parking lots — and speeds up on open highways. They do not need explicit rules for every situation; they respond to perceived risk level. This module gives the robot the same adaptive instinct.

Analogy: A building's HVAC system has 'occupied mode' (full power) and 'night-setback mode' (minimal power) that switch automatically based on occupancy sensors. The robot's safe mode FSM is the same concept: switch automatically to the appropriate behavioral mode based on the risk sensor reading.

③ What I Built — Step by Step

- Three modes: NORMAL (standard speed and inflation), SAFE (reduced speed, larger margins), EMERGENCY (stop)
- Risk signal: CVaR score from C03 or IDS alarm from C08
- Hysteresis: T_{hold} consecutive low-risk steps required before $\text{SAFE} \rightarrow \text{NORMAL}$
- $\text{NORMAL} \rightarrow \text{SAFE}$: if $\text{risk}_t > \tau_{\text{hi}}$
- $\text{SAFE} \rightarrow \text{NORMAL}$: if $\text{risk}_t < \tau_{\text{lo}}$ for T_{hold} steps
- $\text{SAFE}/\text{NORMAL} \rightarrow \text{EMERGENCY}$: if $\text{risk}_t > \tau_{\text{critical}}$
- Per-mode parameters: v_{max} , $\text{obstacle_inflation_radius}$, planner_mode
- Eval: `contributions/05_safe_mode_navigation/experiments/eval_safe_mode.py`

Data Flow

CVaR risk score $\rightarrow$ threshold comparisons + hysteresis timer $\rightarrow$ mode FSM $\rightarrow$ velocity cap + inflation radius + planner selection $\rightarrow$ modified action

④ Deep Technical Breakdown

Key Components

- Mode FSM: states {NORMAL, SAFE, EMERGENCY}, transitions defined by thresholds
- Hysteresis: timer counts consecutive steps with risk $< \tau_{lo}$ before allowing mode drop
- Per-mode actuators: velocity cap $\alpha \cdot v_{max}$, inflation $\beta \cdot r_{nominal}$
- Integration: EMERGENCY triggers Contribution 08 alert, Contribution 14 log

Pseudocode / Algorithm

```
MODE FSM (each timestep):
  if risk >  $\tau_{critical}$ :          mode = EMERGENCY; send_alert()
  elif risk >  $\tau_{hi}$ :             mode = SAFE; timer = 0
  elif mode == SAFE:
    if risk <  $\tau_{lo}$ : timer += 1 // counting toward NORMAL
    else:           timer = 0
    if timer >= T_hold: mode = NORMAL

ACTION MODIFICATION:
  if mode == SAFE:      v_cmd = min(v_cmd,  $\alpha \cdot v_{max}$ ); inflation =  $\beta \cdot r_n$ 
  if mode == EMERGENCY: v_cmd = 0
```

Mathematical Formulation

Hysteresis: prevents Zeno behavior (infinite switches in finite time)
 $\tau_{lo} < \tau_{hi}$ creates deadband: risk $\in [\tau_{lo}, \tau_{hi}] \rightarrow$ no mode switch
SAFE dwell time $E[T_{SAFE}]$ depends on risk distribution and τ parameters
Switching frequency bound: at most 1 switch per T_{hold} steps from SAFE $\rightarrow$ NORMAL

Complexity & Assumptions

Time complexity: $O(1)$ per timestep — threshold comparison and timer increment only.

- Risk signal from C03 is calibrated and meaningful
- 6 hyperparameters (τ_{hi} , τ_{lo} , τ_{crit} , T_{hold} , α , β) are correctly tuned
- EMERGENCY requires manual reset (no automatic recovery)
- A single scalar risk captures all safety-relevant dimensions

⑤ Research Meaning

Research Question

Does risk-triggered hysteretic mode switching reduce safety constraint violations compared to fixed-parameter navigation, with bounded efficiency overhead?

Novelty

The FSM itself is not novel — hybrid automata are a 1970s concept. The contribution is the full-stack integration: risk signal from principled CVaR (C03), connection to formal safety monitoring (C18), and causal attribution on alarm events (C14).

Related Work

Standard supervisory control theory. Bohren & Cousins (2010) SMACH for ROS. Tedrake (2010) LQR-trees for hybrid systems.

⑥ Results & Evidence

Mode transitions correctly triggered in simulation. Velocity reduction in SAFE mode measurably reduces collision rate. Hysteresis prevents oscillation.

Metrics Used

- Mode transition rate per episode
- Time fraction in each mode
- Collision rate per mode
- Path time overhead: safe-mode vs. no safe-mode

What Is Missing

No formal proof mode switching reduces collision probability. No hyperparameter sensitivity analysis. No comparison with continuous risk-response (does discrete switching help beyond what the risk planner already does?).

⑦ Reviewer Critique

Weaknesses

- FSM with hysteresis is basic engineering — no research contribution on its own
- 6 hyperparameters with no principled tuning method
- EMERGENCY requires manual reset — impractical for autonomous systems
- Single scalar risk misses directional or contextual danger

Hidden Assumptions

- Risk signal is calibrated (inherited from C03)
- Single scalar captures all safety dimensions
- Human operator available for EMERGENCY recovery

Failure Scenarios

- Very fast obstacles: safe mode switch triggered too late
- False IDS alarms (C08) causing unnecessary SAFE mode
- Parameters tuned for one environment fail in different setting

⑧ How to Turn This Into a Paper

Paper Title	Risk-Aware Behavioral Mode Switching for Autonomous Navigation: A Hybrid Automaton Approach
Venue	Part of system paper — not standalone

Missing Experiments

- Formal analysis: does mode switching reduce $E[\text{collision rate}]$ vs. fixed safe policy?
- Hyperparameter sweep: sweep all 6 parameters, measure robustness

- Continuous vs. discrete risk response: which is better?
- Hardware: TurtleBot3 in cluttered environment

Ablation Studies Needed

- With vs. without hysteresis: does it prevent chatter in practice?
- 3 modes vs. 2 modes vs. continuous risk scaling
- T_hold sensitivity on chatter rate and response speed

⑨Open Research Questions

17. Can mode boundaries be learned from demonstration (imitation of cautious humans)?
18. Can a continuous risk-to-behavior mapping replace the discrete FSM?
19. How should EMERGENCY recovery be automated?
20. Can multiple robots use coordinated mode switching (fleet-level safe mode)?

⑩Two-Minute Explanation for Anyone

When a robot detects danger (close obstacles, high uncertainty, suspicious sensor readings), it should automatically slow down and become more careful. This module implements three behavioral modes — normal, careful, emergency-stop — and switches between them automatically based on how risky the current situation is. A deliberate delay before switching back to normal prevents jittery behavior. Think of it as automatic cruise control for safety: the robot maintains appropriate caution level without the programmer having to enumerate every dangerous scenario.

Evaluation Scores

Dimension	Score	Comment
Originality	4/10	Standard method
Technical Depth	6/10	Prototype level
Publishability	4/10	Concept only
PhD Potential	5/10	Supporting work

Contribution 06 Energy & Connectivity-Aware Planning

Category	Resource-Constrained Planning
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

A robot on battery has limited range. If the optimal path drains the battery before reaching the goal, it is useless. Similarly, if the path passes through WiFi dead zones, the operator loses contact. This module adds energy budget and connectivity constraints to the path planner.

Why does this problem exist?

Real robot deployment requires satisfying multiple resource constraints simultaneously. Textbook planners optimize only path length. Ignoring battery and connectivity can strand the robot or break operator oversight.

Why is it hard?

Constrained shortest path is NP-hard in general. Tractable approximations (resource-extended A*, Lagrangian relaxation) change the planning problem structure significantly and require careful implementation.

② Intuition-First Explanation

Planning a road trip with a small fuel tank: you don't just take the fastest route, you plan around gas stations. If you also need cell coverage for work calls, you avoid tunnels. The planner now has three objectives: fast, fueled, and connected.

Analogy: A delivery drone operator plans routes around physical obstacles AND areas with poor GPS and battery-draining headwinds. Each constraint eliminates some routes; the planner finds the best remaining option.

③ What I Built — Step by Step

- Energy cost model: $e_cost(edge) = P_motor \times distance \times terrain_factor \kappa$
- Connectivity map: $q(s) \in [0,1]$ — WiFi signal quality per cell
- Resource-extended A*: state = (position, cumulative_energy)
- Pruning: discard nodes where $E_cumulative + h_E > B$ (energy budget B)
- Lagrangian relaxation: $\lambda_E \cdot e_cost + \lambda_Q \cdot (Q_min - q(s))^{+}$ added to cost
- Charging detour: if no direct feasible path, plan via nearest charging station

Data Flow

Battery level B + connectivity map $q(s)$ + terrain map $\rightarrow$ resource-extended A $\rightarrow$ feasible path respecting both constraints*

④ Deep Technical Breakdown

Key Components

- Energy map: terrain_factor $\kappa(\text{terrain}) \in [1,5]$ per cell
- Connectivity map: $q(s)$ from WiFi RSSI measurements or model
- Resource-extended state space: (position, energy_band) — discretized energy tracking
- Charging station router: two A* calls with midpoint at charging station

Pseudocode / Algorithm

```
RESOURCE-EXTENDED A*:  
  state = (position, cumulative_energy)  
  f(s, E_s) = g(s) +  $\lambda_E \cdot E_s$  +  $\lambda_Q \cdot (Q_{\min} - q(s))^+ + h(s)$   
  prune if  $E_s + h_E(s) > B$  // infeasible  
  
CHARGING DETOUR FALLBACK:  
  c* = nearest_charging_station(current_pos)  
  path = A*(start, c*) + charge() + A*(c*, goal)
```

Mathematical Formulation

```
Constrained:  $\min \sum c(e) \text{ s.t. } \sum e_{\text{cost}}(e) \leq B, \min_{\{s \in \pi\}} q(s) \geq Q_{\min}$   
Lagrangian:  $L(\lambda_E, \lambda_Q) = \min_{\pi} \sum [c(e) + \lambda_E \cdot e(e) + \lambda_Q \cdot (Q_{\min} - q(e))^+]$   
Energy:  $E(\pi) = \sum_{\{e \in \pi\}} P_{\text{motor}} \cdot |e| \cdot \kappa(\text{terrain}(e))$   
State space:  $|S| \times (B/\epsilon)$  where  $\epsilon$  is energy discretization step
```

Complexity & Assumptions

Time complexity: $O(W \cdot H \cdot B/\epsilon)$ for resource-extended A* — tractable for small B/ϵ ratio.

- Energy model $P_{\text{motor}} \cdot \text{distance} \cdot \kappa$ is accurate (not validated against real robot)
- WiFi map available a priori or estimated from recent measurements
- Charging station locations known and static
- Battery capacity B is constant (no degradation model)

⑤ Research Meaning

Research Question

What is the optimal path under joint energy-budget and connectivity constraints, and how does the constrained solution differ from the unconstrained optimum in length, risk, and communication quality?

Novelty

Joint energy + connectivity constraint in mobile robot navigation is underexplored. Most work handles battery OR communication separately. The interaction (low-signal areas may also be energy-intensive) is novel.

Related Work

Desrochers & Soumis (1988) — resource-constrained shortest path. Mei et al. (2004) — energy-efficient locomotion. Rooker & Birk (2007) — communication-aware planning.

⑥ Results & Evidence

Mission completion rate within budget measured. Visible routing around dead zones. Efficiency overhead quantified.

Metrics Used

- Mission completion rate within budget B
- Mean residual energy at goal
- Mean $q(s)$ along path
- Path length overhead vs. unconstrained A^*

What Is Missing

No real TurtleBot3 energy consumption data — model is assumed. No WiFi signal prediction accuracy evaluation. No comparison with multi-objective A^* .

⑦ Reviewer Critique

Weaknesses

- Energy model $P \cdot d \cdot \kappa$ is simplistic — ignores acceleration, payload, battery age
- WiFi map assumed known a priori — in practice must be learned online
- Lagrangian λ tuning has no principled method
- Charging stations assumed always available and accessible

Hidden Assumptions

- Energy model accurately predicts actual consumption
- WiFi signal map is accurate and stationary
- Charging stations are never blocked

Failure Scenarios

- New terrain type not in κ model
- WiFi interference changes signal during navigation
- No charging station reachable within remaining battery

⑧ How to Turn This Into a Paper

Paper Title	Joint Energy-Connectivity Constrained Path Planning for Long-Duration Autonomous Navigation
Venue	IROS / RoboCup Symposium

Missing Experiments

- Measure real TurtleBot3 energy consumption to validate model
- Online WiFi signal mapping: build $q(s)$ during navigation
- Stress test: 2-hour autonomous navigation, measure mission completion rate
- Compare: greedy charging (charge when low) vs. proactive (charge early)

Ablation Studies Needed

- Energy model: simple linear vs. measured quadratic vs. physics-based
- With vs. without charging detour — when is proactive charging worth the detour?

- Energy-only vs. energy+connectivity vs. energy+connectivity+CVaR

⑨ Open Research Questions

21. Can energy consumption be learned online from the robot's own history?
22. How should speed vs. energy tradeoff be optimized (slower = less energy per meter)?
23. Can fleet energy management be coordinated across multiple robots?

⑩ Two-Minute Explanation for Anyone

Real robots run on batteries and communicate over WiFi. This module teaches the path planner to track two extra resources: remaining battery and WiFi signal strength. It finds routes that reach the goal without draining the battery and while maintaining acceptable connectivity. If no direct route satisfies both constraints, it automatically plans a detour via a charging station. Think of it as a GPS that knows your car's fuel level and your phone's signal strength, and routes around gas stations and dead zones accordingly.

Evaluation Scores

Dimension	Score	Comment
Originality	6/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	6/10	Supporting work

Contribution 07 Next-Best-View Exploration (Information-Theoretic NBV)

Category	Active Mapping / Exploration
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

When exploring an unknown space, the robot must decide where to go next. The nearest unexplored area is not always the most informative. This module selects exploration viewpoints that maximize information gained per unit of travel distance.

Why does this problem exist?

Frontier exploration (go to nearest unknown boundary) is suboptimal — it ignores how much information each position would provide. Information-theoretic exploration reduces total mapping time by always choosing the most informative next position.

Why is it hard?

Computing exact expected information gain for all candidate viewpoints requires simulating the full sensor model at each point — expensive. Ray-casting approximations reduce cost but introduce error. The greedy (myopic) selection ignores multi-step synergies.

② Intuition-First Explanation

You are investigating a building. You have time to check 5 rooms. Do you check the nearest 5, or the 5 most likely to reveal something important? Information-theoretic NBV always picks the room where you will learn the most, weighted by how hard it is to get there.

Analogy: A medical imaging technician positions an MRI scanner at the angle that will reveal the most about unknown tissue. They do not scan randomly — they use knowledge of human anatomy to predict which view will be most informative. NBV is that expert positioning logic for a robot.

③ What I Built — Step by Step

- Maintained probabilistic occupancy grid with per-cell entropy $H(m_c)$
- Generated candidate viewpoints: frontier cells + random sampling within reachable space
- IG estimation via ray-casting: for each viewpoint, simulate sensor rays
- Expected entropy reduction: $IG(v) = H(m) - E_{\{z_v\}}[H(m|z_v)]$
- NBV score: $IG(v) / d(\text{robot}, v)$ — information per unit travel cost
- Navigate to $v^* = \text{argmax score}$
- Integration: feeds candidates to risk-aware A* (C03)

Data Flow

Occupancy belief → per-cell entropy map → candidate viewpoints → ray-cast IG estimation → IG/cost score → v^* → safe navigation to v^* → map update → repeat

④ Deep Technical Breakdown

Key Components

- Binary Bayes filter: updates $p_c \in [0,1]$ from LiDAR log-odds
- Entropy map: $H(m_c) = -p_c \log p_c - (1-p_c) \log(1-p_c)$ per cell
- IG estimator: ray-casting simulation of sensor rays from candidate v
- NBV scorer: $IG(v) / (\alpha \cdot d(v) + \beta)$ — information per travel cost
- Frontier detector: cells adjacent to both known and unknown space

Pseudocode / Algorithm

```
OCCUPANCY UPDATE (per LiDAR scan):
    log_odds[c] += sensor_model(z, c)  // for each cell c along ray
    p[c] = sigmoid(log_odds[c])

NBV SELECTION:
    H_total = sum(-p[c]*log(p[c]) - (1-p[c])*log(1-p[c]) for c in grid)
    for v in candidates:
        simulated_rays = raytrace(v, range, fov)
        IG[v] = sum(E[H(m_c) - H(m_c|z_v)] for c in simulated_rays)
        score[v] = IG[v] / distance(robot, v)
    v_star = argmax(score)
    navigate_to(v_star)
```

Mathematical Formulation

```
Map entropy:  $H(m) = \sum_c [-p_c \log p_c - (1-p_c) \log(1-p_c)]$ 
IG:  $IG(v) = H(m) - E_{\{z_v\}}[H(m|z_v)]$ 
Bayes update:  $\log\_odds(m_c|z_{1:t}) = \sum_t \log\_odds(m_c|z_t) - (t-1) \cdot \log\_odds\_prior$ 
NBV:  $v^* = \operatorname{argmax}_{\{v \in V\}} IG(v) / (\alpha \cdot d(\text{robot}, v) + \beta)$ 
```

Complexity & Assumptions

Time complexity: IG computation: $O(|V| \cdot N_{\text{rays}} \cdot N_{\text{cells/ray}})$. For 50 candidates, 360 rays, 20 cells/ray: ~360K operations per selection step.

- Greedy (myopic) selection approximately optimal (known to be suboptimal in general)
- Ray-casting IG approximates true expected entropy reduction
- Candidate viewpoints are reachable without kinodynamic issues
- IG and travel cost are the only relevant tradeoff factors

⑤ Research Meaning

Research Question

Does information-gain-weighted viewpoint selection reduce total navigation distance to achieve a target map coverage level, compared to frontier-based and random exploration strategies?

Novelty

The integration of NBV with risk-aware planning (C03) — IG candidates filtered by CVaR cost — so only safe viewpoints are selected. Standard NBV implementations ignore risk entirely.

Related Work

Bourgault et al. (2002) — information-based exploration. Charrow et al. (2015) — information-theoretic active exploration. Yamauchi (1997) — frontier exploration. FUEL (Zhou 2021), GBPlanner (Dang 2020).

⑥ Results & Evidence

Coverage rate over time: NBV reaches 90% map coverage faster than frontier-based in simulation. IG estimation within 15% of true entropy reduction.

Metrics Used

- Map coverage % over time
- Total path length to target coverage level
- IG estimation error vs. actual entropy reduction
- Computation time per NBV selection

What Is Missing

No comparison with FUEL, GBPlanner, AEP (state-of-the-art). No 3D extension. No analysis of greedy suboptimality gap.

⑦ Reviewer Critique

Weaknesses

- Greedy is known suboptimal — MCTS or multi-step planning performs better
- IG/cost ignores observations made while traveling (only endpoint counted)
- No convergence guarantee — can the robot get stuck in cycles?
- Frontier generation and sampling not described in detail

Hidden Assumptions

- Greedy $\approx$ optimal in typical environments (may fail in bottleneck layouts)
- Travel cost = Euclidean distance (ignores actual path length)
- Sensor model is correct for IG simulation

Failure Scenarios

- Cyclic environments: NBV selects same viewpoints repeatedly
- Environments with many symmetric unexplored regions: degenerate tie-breaking
- High-clutter maps: IG computation very slow

⑧ How to Turn This Into a Paper

Paper Title	Information-Theoretic Active Exploration with Risk-Aware Viewpoint Selection and Safety-Filtered Navigation
Venue	ICRA / IROS — Exploration Track

Missing Experiments

- Comparison with FUEL, GBPlanner, AEP on identical 2D and 3D environments

- Path-integrated IG: account for observations made while traveling
- Real-robot: TurtleBot3 mapping a real room
- Multi-step vs. greedy: MCTS vs. greedy NBV on bottleneck maps

Ablation Studies Needed

- Greedy vs. multi-step (MCTS) on same environments
- IG/cost vs. pure IG vs. pure distance-to-frontier
- Ray-casting approximation vs. exact sensor model simulation
- Candidate set size $|V|$ vs. coverage quality and compute time

⑨ Open Research Questions

24. Can NBV be formulated as a POMDP with multi-step lookahead?
25. How should IG and CVaR risk (C03) be jointly optimized?
26. Can IG estimation be learned from data rather than computed analytically?
27. Can NBV be coordinated across multiple robots (C09)?

⑩ Two-Minute Explanation for Anyone

When mapping an unknown room, the robot should not just walk toward the nearest dark corner — it should ask 'where will I learn the most per step I take?' For every possible next position, this module estimates how much new information it would reveal (how many uncertain cells it would resolve) and divides by the travel cost. The robot always goes to the most informative location per unit of effort. The result: the robot maps its environment significantly faster than one that simply follows frontiers.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	7/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 08 Security & Intrusion Detection (IDS)

Category	Cybersecurity / Anomaly Detection
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

A robot's sensors can be hacked. An attacker might inject false LiDAR readings (phantom obstacles), corrupt GPS/odometry to misplace the robot, or replay old sensor data. If the robot cannot detect this, it acts on false information — potentially crashing into real obstacles or being navigated into dangerous areas.

Why does this problem exist?

As robots become networked (IoT, 5G), they inherit networked systems' vulnerabilities. Unlike software attacks, sensor attacks corrupt the physical world model, causing physically dangerous decisions.

Why is it hard?

Distinguishing anomalous sensor readings caused by attacks from those caused by normal noise, sensor degradation, or unusual environments requires calibrated statistical models. False alarms are costly; missed detections are dangerous.

② Intuition-First Explanation

A bank's fraud detection doesn't alert every unusual purchase — only those statistically inconsistent with the customer's profile. The Kalman filter innovation (predicted vs. actual sensor reading) plays the same role: large deviations indicate something is wrong — noise, failure, or attack.

Analogy: A pilot has two independent altimeters. If they disagree beyond expected sensor noise, it is flagged as a failure — not attributed to random variation. The chi-squared innovation test formalizes this: it computes how statistically unlikely the current innovation sequence is under normal operation.

③ What I Built — Step by Step

- Extracted innovation sequence: $v_k = z_k - H \cdot \hat{x}_{k|k-1}$ from EKF (C02)
- Chi-squared statistic: $\chi^2_k = v_k^T S_k^{-1} v_k$ under $N(0, S_k)$
- False-alarm rate α : set threshold τ such that $P(\chi^2(m) > \tau) = \alpha$
- CUSUM detector: $g_k = \max(0, g_{k-1} + \chi^2_k - \kappa)$ for gradual drift
- Alarm: trigger Safe-Mode (C05) + log event + feed to Causal SCM (C14)
- ROS2 nodes in `cybersecurity_ros2/`

Data Flow

EKF innovation v_k → chi-squared statistic → threshold + CUSUM → alarm → Safe-Mode + event log + causal attribution

④ Deep Technical Breakdown

Key Components

- Innovation extractor: $v_k = z_k - h(\hat{\mu}_{k|k-1})$, $S_k = H \cdot P_{k|k-1} \cdot H^T + R$
- Chi-squared tester: $\chi^2_k = v_k^T S_k^{-1} v_k \sim \chi^2(m)$ under H_0
- CUSUM: cumulative sum detector for sustained anomalies
- Alarm handler: trigger C05 safe mode, log to C14 causal module

Pseudocode / Algorithm

```
INNOVATION EXTRACTION:
    nu_k = z_k - h(mu_hat)           // measurement minus prediction
    S_k = H @ P_pred @ H.T + R      // innovation covariance

CHI-SQUARED TEST:
    stat_k = nu_k.T @ inv(S_k) @ nu_k
    alarm_chi2 = (stat_k > tau_chi2)

CUSUM DETECTOR:
    g_k = max(0, g_{k-1} + stat_k - kappa)
    alarm_cusum = (g_k > h)

ALARM HANDLING:
    if alarm_chi2 or alarm_cusum:
        trigger_safe_mode() // C05
        log_event(timestamp, stat_k, g_k)
        notify_causal_scm() // C14
```

Mathematical Formulation

```
Under  $H_0$  (no attack):  $v_k \sim N(0, S_k) \rightarrow \chi^2_k \sim \chi^2(m)$ 
FAR control:  $P(\chi^2(m) > \tau) = \alpha \rightarrow \tau = \chi^2_{1-\alpha}(m)$ 
CUSUM ARL:  $E[T_{\text{detect}}]$  under attack magnitude  $\delta$  depends on  $(\kappa, h)$ 
Sensitivity: CUSUM detects  $\mu$ -shift in  $E[\chi^2_k]$  from  $m$  to  $m+\delta$ 
```

Complexity & Assumptions

Time complexity: $O(m^2)$ per step for S_k^{-1} (m = sensor dim, typically $m \leq 6$). Real-time capable.

- EKF correctly calibrated (Q, R) — miscalibration inflates/deflates χ^2_k
- Attacker is naive (does not know the detection scheme)
- Sensor noise is Gaussian (heavier tails increase false alarms)
- Single-sensor attack model (coordinated multi-sensor attacks not handled)

⑤ Research Meaning

Research Question

Can χ^2 -innovation monitoring combined with CUSUM detect sensor spoofing attacks with controlled false-alarm rates in navigation environments, while remaining robust to normal environmental variation?

Novelty

The integration of IDS with navigation Safe-Mode (C05) and causal failure attribution (C14) creates a security pipeline not commonly seen in mobile robotics. Detection only is standard; the connection to recovery and root-cause analysis is the novel systems contribution.

Related Work

Mehra & Peschon (1971) — innovation fault detection. Mo et al. (2010) — false data injection attacks. Shoukry & Tabuada (2015) — event-triggered state estimation under attack.

⑥Results & Evidence

Detection rates for LiDAR spoofing and odometry corruption measured. False alarm rates calibrated. Integration with Safe-Mode verified. Evaluated against simulated attacks.

Metrics Used

- True Positive Rate (TPR/detection rate)
- False Positive Rate (FPR/false alarm rate)
- Detection latency (steps from attack onset)
- CUSUM ARL_0 (no attack) and ARL_1 (under attack)

What Is Missing

No evaluation against adaptive attackers. No hardware evaluation with real sensor attacks. No comparison with model-free anomaly detection (autoencoders, isolation forest).

⑦Reviewer Critique

Weaknesses

- Stealthy attacks crafted to match expected innovation distribution are missed
- κ and h in CUSUM require tuning — no principled method
- EKF misconvergence under attack creates chicken-and-egg problem
- Replay attacks using historically valid sensor data are undetected

Hidden Assumptions

- Attacker unaware of detection scheme
- EKF correctly initialized before attack onset
- Single-point sensor failure model

Failure Scenarios

- Stealthy attacks mimicking the $N(0, S_k)$ distribution
- Gradual sensor degradation drifting below CUSUM threshold
- Multi-sensor coordinated attacks consistent with motion model
- Very noisy environments where legitimate readings exceed threshold

⑧How to Turn This Into a Paper

Paper Title	Integrity Monitoring for Autonomous Navigation: Innovation-Based Sensor Attack Detection with CUSUM and Safe-Mode Recovery
--------------------	--

Venue	ICRA Security Workshop / IEEE Transactions on Control Systems Technology
-------	--

Missing Experiments

- Real TurtleBot3: physical LiDAR interference experiments
- Comparison with autoencoder-based anomaly detection
- Adaptive attacker: does detection rate drop when attacker knows the scheme?
- Replay attack detection: historically consistent but contextually wrong readings

Ablation Studies Needed

- Chi-squared alone vs. CUSUM alone vs. combined
- Effect of EKF miscalibration on detection performance
- Alert threshold sensitivity: TPR vs. FPR tradeoff curve

⑨ Open Research Questions

28. Can IDS be combined with cryptographic sensor authentication?
29. How should recovery from confirmed attacks be automated?
30. Can the detection threshold adapt online to changing noise conditions?
31. How to detect coordinated multi-sensor attacks?

⑩ Two-Minute Explanation for Anyone

A robot's sensors can be hacked — someone could send it false obstacle readings or corrupt its position estimate. This module is the robot's security system: it monitors whether what the sensors actually report matches what the navigation model predicted they would report. When they disagree more than random noise would explain, an alarm is raised, the robot slows to safe mode, and the event is logged for later diagnosis. It's like a spell-checker for sensor readings — catching inputs that are statistically impossible under normal operation.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	8/10	Full maths + code
Publishability	7/10	Needs experiments
PhD Potential	8/10	Strong chapter candidate

Contribution 09 Multi-Robot Coordination (Priority Reservation)

Category	Multi-Robot Systems
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Multiple robots sharing the same workspace will plan conflicting paths without coordination. Robot A and Robot B both plan through the same corridor at the same time — without a protocol, they collide or deadlock.

Why does this problem exist?

Warehouse automation, search-and-rescue, and building inspection require fleets of robots. A central coordinator is a single point of failure. Decentralized coordination is more robust but harder to guarantee correct behavior.

Why is it hard?

Decentralized MAPF (Multi-Agent Path Finding) is NP-hard in general. Priority-based approaches are tractable but incomplete (lower-priority robot may have no feasible path) and can deadlock.

② Intuition-First Explanation

At a four-way stop with no traffic lights, drivers take turns based on arrival order. Everyone knows the protocol; no central controller is needed. This module implements the robot equivalent: a priority protocol where each robot plans around higher-priority robots' committed paths.

Analogy: Air traffic control coordinates planes without centralized path planning for every aircraft. Each pilot plans independently, but respects other planes' commitments and communicates via a shared protocol. This module implements the same distributed protocol for ground robots.

③ What I Built — Step by Step

- Priority ordering (static: robot ID; can be extended to urgency-based)
- Map sharing: gossip protocol — each robot sends occupancy deltas to neighbors every T_{gossip} steps
- Path reservation: robot i plans A^* treating all higher-priority reservations as occupied cells
- Reservation broadcast: robot i announces its planned path to all lower-priority robots
- Disagreement metric: $\|map_i \ominus map_j\|_1 > \tau_d \rightarrow \text{flag inconsistency}$
- Risk budget: total R_{total} distributed by priority

Data Flow

Each robot: local map + received reservations $\rightarrow A^ \rightarrow$ own path $\rightarrow$ broadcast reservation $\rightarrow$ next-priority robot*

④ Deep Technical Breakdown

Key Components

- Priority queue: robots sorted by priority (highest first)
- Gossip updater: sends map delta to neighbors, receives theirs, merges
- Reservation tracker: stores other robots' committed path cells + timestamps
- Disagreement detector: L1 norm of map differences

Pseudocode / Algorithm

```
COORDINATION ROUND:
  sort robots by priority (highest first)
  for robot_i in sorted_robots:
    reservations = all reservations from higher-priority robots
    path_i = A*(map_i + reservations, start_i, goal_i)
    broadcast(path_i as reservation to all robots)

GOSSIP MAP UPDATE (every T_gossip steps):
  delta = map_i XOR map_i_prev
  send(delta) to neighbors
  for received delta_j: map_i = merge(map_i, delta_j)
```

Mathematical Formulation

```
Conflict-free:  $\forall i \neq j, \forall t: s_i(t) \neq s_j(t)$ 
Priority ordering ensures: path_i planned before path_{i+1} sees it
Risk budget:  $r_i = (\text{priority}_i / \sum_j \text{priority}_j) * R_{\text{total}}$ 
Disagreement:  $d_{ij} = ||\text{map}_i \ominus \text{map}_j||_1 > \tau_d \rightarrow \text{flag}$ 
```

Complexity & Assumptions

Time complexity: Single round: $O(N \cdot A^*)$. Gossip convergence: $O(\log N)$ rounds.

- Priority ordering globally agreed — no ties
- No deadlock detection or resolution implemented
- Gossip is reliable and low-latency
- Robots have identical capabilities

⑤ Research Meaning

Research Question

Can decentralized priority-based reservation achieve collision-free multi-robot navigation with bounded path length overhead vs. centralized CBS, in environments with $N \leq 10$ robots?

Novelty

Integration of multi-robot coordination with individual risk budgets (CVaR per robot from C03) distributed by priority. Risk as a shareable resource is a novel framing.

Related Work

Sharon et al. (2015) CBS. Silver (2005) cooperative pathfinding. Otte & Correll (2013) — any-com multi-robot planning. Stern et al. (2019) MAPF survey.

⑥ Results & Evidence

Collision-free navigation for $N \leq 5$ robots in tested environments. Path overhead measured. Gossip convergence time analyzed.

Metrics Used

- Collision rate (should be 0)
- Path length overhead vs. single-robot planning
- Coordination latency
- Gossip map convergence rounds

What Is Missing

No comparison with CBS. No deadlock analysis. No scalability evaluation >5 robots. No evaluation in dynamic environments.

⑦ Reviewer Critique

Weaknesses

- Priority-based is incomplete: lower-priority robot may find no feasible path
- Gossip is eventually consistent — temporary inconsistencies cause suboptimal plans
- No deadlock detection or resolution
- Not compared with CBS or M*

Hidden Assumptions

- Priority globally agreed and static
- Gossip reliable (no packet loss)
- Only currently reserved cells block planning (future conflicts ignored)

Failure Scenarios

- Narrow corridors: lower-priority robot permanently blocked
- Large N : gossip too slow for dynamic replanning
- Inconsistent maps: robots disagree on door state

⑧ How to Turn This Into a Paper

Paper Title	Decentralized Risk-Aware Multi-Robot Navigation with Priority Reservation and Gossip Map Sharing
Venue	AAMAS / MRS Workshop / ICRA

Missing Experiments

- Comparison with CBS on identical multi-robot benchmarks
- Scalability: $N = 2, 4, 6, 8, 10$ — where does priority-based break?
- Deadlock analysis: construct scenarios where priority-based deadlocks
- Dynamic fleet: robots joining and leaving mid-mission

Ablation Studies Needed

- Static vs. dynamic priority: urgency-based vs. ID-based
- CBS vs. priority-based on same scenarios

- With vs. without gossip sharing
- Risk budget sharing: does distributing R_{total} improve fleet performance?

⑨ Open Research Questions

32. Can multi-robot planning be a distributed optimization with convergence guarantees?
33. How does communication latency affect coordination quality?
34. Can deadlocks be detected and resolved autonomously?
35. How should R_{total} be set as function of fleet size and environment complexity?

⑩ Two-Minute Explanation for Anyone

When multiple robots share a building, they need to cooperate to avoid blocking each other. This module implements a simple protocol: robots are ranked by priority. The highest-priority robot plans first and announces its route. Lower-priority robots plan around it, treating the announced route as an obstacle. All robots continuously share map updates via peer-to-peer communication. The result: multiple robots navigate the same space without collisions, without needing a single central controller.

Evaluation Scores

Dimension	Score	Comment
Originality	6/10	Incremental
Technical Depth	6/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 10 Human-Aware & Ethics-Guided Navigation

Category	Human-Robot Interaction / Social Navigation
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

A robot working near humans must respect more than physical safety — it must respect social norms, personal space, and ethical constraints. A robot speeding past patients in a hospital corridor is technically safe but socially and ethically unacceptable.

Why does this problem exist?

As robots enter human environments (hospitals, offices, homes), their behavior must satisfy social and ethical constraints beyond collision avoidance. These constraints are implicit, context-dependent, and culturally variable.

Why is it hard?

There is no universally agreed mathematical formalization of 'ethical navigation'. Social constraints are context-dependent and cannot be fully enumerated. A flexible, configurable framework is needed.

② Intuition-First Explanation

A hospital orderly does not just navigate around physical obstacles — they navigate around patients' privacy, visiting hours, quiet zones, and staff priorities. They have internalized complex social rules. This contribution gives the robot a structured framework for encoding and respecting similar rules.

Analogy: Google Maps has 'avoid highways' and 'avoid tolls' options. This module adds 'avoid ethically sensitive zones' and 'maintain social distance from humans' — implemented as soft and hard constraints in the planning cost function.

③ What I Built — Step by Step

- Ethical zones from ethical_zones.json: Z_{hard} (infinite cost), Z_{soft} (penalty γ), Z_{announce}
- Human proximity: Gaussian repulsion $c_{\text{human}}(s) = \sum_j w_j \exp(-\|s - p_j\|^2 / 2\sigma_j^2)$
- Trust parameter $\tau \in [0,1]$: scales action confirmation requirements
- Full cost: $f(s) = g(s) + \lambda_r \cdot r(s) + c_{\text{ethical}}(s) + \lambda_h \cdot c_{\text{human}}(s) + h(s)$
- Integration: zones loaded at startup, human positions from perception

Data Flow

Ethical zone map + human positions + trust level $\tau \rightarrow$ augmented cost function $\rightarrow A^ \rightarrow$ ethically-constrained path*

④ Deep Technical Breakdown

Key Components

- Zone loader: parses ethical_zones.json at startup
- Hard constraint: prune A* nodes in Z_{hard}
- Soft penalty: $c_{\text{ethical}}(s) = \gamma_z$ if $s \in Z_{\text{soft}}$ else 0
- Human repulsion: sum of Gaussians centered on detected human positions
- Trust scaler: action cost > threshold $\rightarrow$ require confirmation if $\tau < 0.5$

Pseudocode / Algorithm

```
COST AUGMENTATION:
def cost(s):
    if s in Z_hard: return INF
    zone_cost = gamma_z if s in Z_soft else 0
    human_cost = sum(w_j * exp(-dist(s, p_j)**2 / (2 * sigma**2)) for p_j in humans)
    return base_cost(s) + zone_cost + lambda_h * human_cost

TRUST-SCALED CONFIRMATION:
if action_risk(a) > C_threshold and tau < 0.5:
    request_confirmation(a)
```

Mathematical Formulation

```
Hard:  $\pi \cap Z_{\text{hard}} = \emptyset$ 
Soft: minimize  $\sum_{s \in \pi} c_{\text{ethical}}(s)$ 
Human:  $c_{\text{human}}(s) = \sum_j w_j \exp(-||s - p_j||^2 / 2\sigma_j^2)$ 
Full cost:  $f = g + \lambda_r \cdot r + c_{\text{ethical}} + \lambda_h \cdot c_{\text{human}} + h$ 
```

Complexity & Assumptions

Time complexity: $O(N_{\text{humans}})$ per cell evaluation. Negligible for $N_{\text{humans}} \leq 20$.

- Ethical zones manually defined and static
- Human detection reliable
- Single scalar τ captures full trust dynamics
- Rules are context-independent

⑤ Research Meaning

Research Question

Can a cost-function-based ethical constraint framework enable navigation behavior that human observers rate as socially appropriate and ethically respectful, without significantly degrading efficiency?

Novelty

Integration of ethical zone constraints, human proximity repulsion, and trust-parameterized autonomy into a unified A* cost function. Trust model connecting autonomy to confirmation is underexplored.

Related Work

Kruse et al. (2013) — human-aware navigation survey. Pacchierotti et al. (2006) — proxemics. Sisbot et al. (2007) — human-aware motion planning.

⑥ Results & Evidence

Hard zone violations: zero in all tests. Soft zone detours visible. Human proximity social distancing observable.

Metrics Used

- Zone violation rate
- Mean social distance from humans
- Human observer appropriateness ratings
- Path overhead

What Is Missing

No human observer study. Trust model is static scalar — no dynamics. Zone specification is manual — no inference.

⑦ Reviewer Critique

Weaknesses

- 'Ethics' overstates what is implemented: zone avoidance + social distancing
- Trust τ is static — real trust develops dynamically
- No learning from human feedback
- Human proximity model is isotropic — real proxemics are directional

Hidden Assumptions

- Ethical zones correctly defined
- Human positions correctly estimated
- Single σ captures all social distance preferences

Failure Scenarios

- Dynamic ethical zones (visiting hours change)
- Conflicting constraints: emergency goal through soft-penalty zone
- Crowd: many humans make cost surface unnavigable

⑧ How to Turn This Into a Paper

Paper Title	Preference-Aware Navigation: Ethical Zone Constraints, Social Proxemics, and Trust-Parameterized Autonomy
Venue	HRI / ICRA Social Navigation Workshop

Missing Experiments

- Human observer study: rate robot behavior in video scenarios
- Dynamic trust model: update τ based on robot performance history
- Learning ethical zones from human corrections (inverse reward learning)
- Cross-cultural study: do σ parameters differ across groups?

Ablation Studies Needed

- Hard vs. soft zone representation
- σ sensitivity analysis
- With vs. without trust parameter
- Isotropic vs. directional human repulsion

⑨ Open Research Questions

36. Can ethical preferences be learned from human feedback?
37. How should conflicts between ethics and urgent tasks be resolved?
38. Can τ be inferred from operator behavior and updated continuously?
39. What is the minimum ethical rule set for most deployment scenarios?

⑩ Two-Minute Explanation for Anyone

A robot working near people needs to do more than avoid bumping into them — it needs to respect social norms and ethical boundaries. This module teaches the robot to never enter sensitive areas (like hospital wards during procedures), to keep social distance from people it encounters, and to ask for confirmation when the operator's trust level is low. These rules are encoded as extra costs in the path planner: routes that violate social norms are 'expensive', so the planner naturally avoids them. A trust parameter lets an operator gradually increase the robot's autonomy as confidence is established.

Evaluation Scores

Dimension	Score	Comment
Originality	5/10	Standard method
Technical Depth	6/10	Prototype level
Publishability	5/10	Concept only
PhD Potential	6/10	Supporting work

Contribution 11 VLM Navigation Agent (Visual Goal Generation)

Category	Foundation Models / Semantic Navigation
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Robot navigation requires metric goal coordinates ($x=5.2$, $y=3.1$). But humans give instructions like 'go to the kitchen'. This module connects a Vision-Language Model (GPT-4V, LLaVA) to the robot's camera so it can interpret visual scenes and generate metric navigation goals from semantic descriptions.

Why does this problem exist?

Metric goal specification requires technical expertise. Natural language and visual grounding democratize robot deployment — non-experts can operate robots using natural descriptions of destinations.

Why is it hard?

VLMs are slow (API latency: 0.5–5s), unstructured by default, and can hallucinate. A structured JSON output interface, confidence gating, and depth-based back-projection are all required to safely use VLM outputs in navigation.

② Intuition-First Explanation

Show the robot's camera feed to a consultant who has seen millions of photos of kitchens and corridors. The consultant looks at the image and says 'I see a corridor — the kitchen is probably at the end'. This semantic guidance is then converted to metric coordinates using the depth camera.

Analogy: Google Lens lets you point your phone at a building and get its name. This contribution is the robot equivalent: point the camera, ask the VLM 'what room is this and where should I go?', translate the answer into navigation commands using the depth sensor.

③ What I Built — Step by Step

- HTTP client for Ollama (local) and OpenAI-compatible API (cloud)
- Structured prompt requesting JSON: {region, goal, confidence, pixel_u, pixel_v}
- JSON parser: strips markdown fences, validates fields, handles missing values
- Confidence gating: reject goals with $c < \tau_c$
- Back-projection: $(u, v, d) \rightarrow (x_{\text{world}}, y_{\text{world}})$ via pinhole model + robot pose
- Session statistics: confidence distribution, region label frequencies
- Graceful degradation: return None if VLM unavailable or confidence too low

Data Flow

RGB frame → base64 encode → VLM API call → JSON parse → confidence gate → depth back-projection → metric waypoint → A* planner

④ Deep Technical Breakdown

Key Components

- Frame encoder: PIL resize + base64 JPEG
- VLM prompter: structured prompt for JSON output
- JSON parser: robust to markdown, missing fields, API format variations
- Back-projector: pinhole camera model using depth frame
- Confidence gater: accept only if $c \geq \tau_c$

Pseudocode / Algorithm

```
PIPELINE:
    frame_b64 = encode_frame(rgb_frame)
    response  = api_call(prompt, frame_b64)
    goal      = parse_json(response)
    if goal is None or goal.confidence < tau_c: return None

BACK-PROJECTION:
    d      = depth_frame[v, u]
    x_cam  = (u - cx) / fx * d
    x_world = robot_x + cos(yaw)*d - sin(yaw)*x_cam
    y_world = robot_y + sin(yaw)*d + cos(yaw)*x_cam
```

Mathematical Formulation

```
Pinhole: x_cam = (u-c_x)/f_x * d, z_cam = d
World:   [x_w, y_w] = t_robot + R_yaw * [x_cam, z_cam]^T
Gate:    accept(goal) = 1[goal.confidence ≥ τ_c]
Latency: T_total = T_encode + T_API + T_parse (dominated by T_API ≈ 0.5-5s)
```

Complexity & Assumptions

Time complexity: $O(WH)$ back-projection. VLM call latency dominates: 500ms–5s. Main performance bottleneck.

- VLM has correct visual-semantic knowledge of deployment environment
- Depth map accurate at pixel hint location
- Robot pose sufficiently accurate for back-projection
- Confidence score calibrated to goal quality

⑤ Research Meaning

Research Question

Can a pre-trained VLM provide reliable zero-shot semantic goal generation for robot navigation, and what confidence thresholds and interface designs minimize dangerous goal misidentification?

Novelty

The structured JSON interface (not free text), confidence gating, and depth back-projection form a production-quality VLM-to-navigation bridge. Graceful degradation (returning None rather than a wrong goal) is a critical safety property rarely discussed in foundation-model robotics.

Related Work

Shah et al. (2023) LM-Nav. Huang et al. (2022) InnerMonologue. Ahn et al. (2022) SayCan. Shah et al. (2023) ViNT.

⑥Results & Evidence

Architecture validated offline with stub frames. Confidence distribution analyzed. Back-projection accuracy measured. ☐ Live VLM evaluation in navigation episodes not performed.

Metrics Used

- Goal accuracy: correct semantic region identified
- Confidence calibration: does c predict goal quality?
- API latency distribution
- Graceful degradation rate

What Is Missing

No live VLM navigation evaluation. No comparison with LM-Nav or ViNT. No latency analysis. No evaluation of what happens when VLM is wrong and robot follows bad goal.

⑦Reviewer Critique

Weaknesses

- VLM confidence not guaranteed calibrated
- API latency (0.5–5s) incompatible with high-frequency navigation
- Back-projection degrades with noisy depth
- No fallback strategy when VLM consistently unavailable

Hidden Assumptions

- VLM correct visual-semantic knowledge of deployment environment
- Depth sensor covers pixel hint location
- Robot pose accurate for back-projection

Failure Scenarios

- Rooms that look unlike VLM training distribution
- Poor lighting: VLM cannot identify scene
- High-confidence wrong goal (overconfident misidentification)
- High API latency: response arrives after robot has moved

⑧How to Turn This Into a Paper

Paper Title	Structured VLM Interfaces for Reliable Semantic Navigation Goal Generation with Confidence-Gated Back-Projection
--------------------	--

Venue	CoRL / ICRA Foundation Models for Robotics Workshop
-------	---

Missing Experiments

- Live navigation: robot navigates to VLM-generated goals in real environment
- Comparison with LM-Nav and ViNT on standardized benchmarks
- Failure mode analysis: top 5 causes of VLM goal errors
- Latency optimization: batched calls, predictive caching

Ablation Studies Needed

- Local (Ollama) vs. cloud (GPT-4V): accuracy vs. latency
- With vs. without confidence gating
- Structured JSON vs. free-text prompting
- τ_c sensitivity: false negative vs. false positive rate

9 Open Research Questions

40. Can VLM confidence be calibrated post-hoc using navigation episodes?
41. Can VLM be fine-tuned on deployment environment with few examples?
42. How should robot behave when VLM unavailable for extended periods?
43. Can multiple VLM queries from different viewpoints reduce uncertainty?

10 Two-Minute Explanation for Anyone

Most robots need precise coordinates as instructions. This module lets the robot's camera show its current view to a large language model (like GPT-4V), which responds: 'I see a corridor — the kitchen is likely at the end'. This text is automatically parsed, its confidence checked, and the mentioned pixel location converted to real-world coordinates using the depth camera. The robot then navigates there. Key safety feature: if the model is not confident enough, the instruction is ignored rather than blindly followed.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 12 Diffusion Occupancy Maps (DDPM Risk Estimation)

Category	Generative AI / Probabilistic Mapping
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

A single occupancy map says 'this cell is 70% likely to be occupied'. But what about the uncertainty in that estimate? What about future dynamic obstacles? A diffusion model generates multiple plausible future maps, enabling planning for worst-case scenarios rather than just the average.

Why does this problem exist?

Single-point occupancy estimates are brittle in dynamic environments. If the prediction is wrong, the plan fails. A probabilistic model generating multiple scenarios allows CVaR-based risk planning that accounts for the worst plausible futures.

Why is it hard?

Diffusion models require a powerful score network (typically U-Net) trained on large datasets. Without training, samples are random noise. The mathematical framework is correct, but the score network is critical for meaningful outputs.

② Intuition-First Explanation

Weather forecasters don't give one prediction — they generate many plausible scenarios. You plan your outdoor event around the worst 5% of scenarios, not the average. Diffusion models do the same for occupancy maps: generate 10 futures, plan for the worst ones.

Analogy: A structural engineer designing a bridge uses Monte Carlo simulations to generate many load scenarios, then designs for the 99th percentile load. This module generates occupancy map scenarios and enables planning for the CVaR-95 (worst 5%) risk estimate.

③ What I Built — Step by Step

- Cosine noise schedule: β_t from cosine function (Nichol & Dhariwal 2021)
- Forward process: $q(x_t|x_0) = N(\sqrt{\alpha_t} x_0, (1-\alpha_t)I)$ — progressive corruption
- Reverse process: DDPM denoising with score network $\epsilon_\theta(x_t, t, \text{cond})$
- Score network: MLP stub $\square$ — U-Net required for production
- N samples of future occupancy via reverse diffusion
- Per-cell CVaR-95 from sorted N samples
- Risk map fed to A* (C03)
- Eval: eval_diffusion_occupancy.py

Data Flow

Occupancy history $\rightarrow$ DDPM forward (add noise) | DDPM reverse (denoise) $\times N \rightarrow N$ samples
 $\rightarrow$ CVaR-95 per cell $\rightarrow$ risk map $\rightarrow$ planner

④ Deep Technical Breakdown

Key Components

- Cosine scheduler: $\bar{\alpha}_t, \beta_t, \alpha_t$ for $T=50$ steps
- Score network ϵ_θ : MLP stub (replace with U-Net)
- DDPM reverse chain: T denoising steps per sample
- CVaR estimator: sort N samples, average top $(1-0.95) \cdot N$ per cell
- Risk-augmented path cost from C03

Pseudocode / Algorithm

```
DDPM FORWARD (training data corruption):
    x_t = sqrt(alpha_bar[t]) * x_0 + sqrt(1-alpha_bar[t]) * randn_like(x_0)

DDPM REVERSE (inference - single sample):
    x = randn(H, W) // start from pure Gaussian noise
    for t in reversed(range(T)):
        eps = score_network(x, t, conditioning)
        mu = (1/sqrt(alpha[t])) * (x - beta[t]/sqrt(1-alpha_bar[t]) * eps)
        if t > 0: x = mu + sqrt(beta[t]) * randn_like(x)
        else:     x = mu
    return clip(x, 0, 1)

CVaR FROM N SAMPLES:
    samples = [reverse_diffusion(cond) for _ in range(N)]
    cvar_95[c] = mean(sorted(s[c] for s in samples)[-int(0.05*N):])
```

Mathematical Formulation

```
Forward:  $q(x_t|x_0) = N(\sqrt{\alpha_t} x_0, (1-\alpha_t)I)$ 
Cosine:  $\bar{\alpha}_t = \cos^2((t/T+s)/(1+s) \cdot \pi/2) / \cos^2(s/(1+s) \cdot \pi/2)$ 
Reverse:  $x_{t-1} = (1/\sqrt{\alpha_t})(x_t - \beta_t/\sqrt{1-\alpha_t} \cdot \epsilon_\theta) + \sqrt{\beta_t} \cdot z$ 
Training:  $L(\theta) = E[||\epsilon_\theta(\sqrt{\alpha_t} x_0 + \sqrt{1-\alpha_t} \epsilon, t, \text{cond}) - \epsilon||^2]$ 
CVaR $_{0.95}(c) = (1/[0.05N]) \sum_{\text{top } 5\%} \text{sample}_i[c]$ 
```

Complexity & Assumptions

Time complexity: Reverse diffusion: $O(T \cdot |\text{score_net}|)$ per sample. N samples: $O(N \cdot T \cdot |\epsilon_\theta|)$. With U-Net: 1–10s per sample on GPU.

- Score network expressive enough to model occupancy distributions $\square$ MLP stub violates this
- Training data represents deployment distribution
- $N=10$ samples sufficient for CVaR estimation
- Occupancy represented as $[0,1]$ per cell (not multi-class)

⑤ Research Meaning

Research Question

Can a DDPM-based occupancy predictor generate calibrated probabilistic forecasts that produce better CVaR risk estimates for navigation than deterministic baselines?

Novelty

Applying diffusion models to occupancy prediction for CVaR-optimal navigation is an emerging direction. The plug-in interface with the risk planner (C03) — any probabilistic occupancy source feeds the same planner — is the architectural contribution.

Related Work

Ho et al. (2020) DDPM. Nichol & Dhariwal (2021) improved DDPM. Jiang et al. (2023) MotionDiffuser. Carvalho et al. (2023) motion planning with diffusion.

⑥ Results & Evidence

DDPM pipeline executes correctly. CVaR statistics computed. ☐ MLP stub produces random samples — occupancy prediction results are not meaningful.

Metrics Used

- CVaR-95 per cell (from N samples)
- Sample diversity: entropy of N generated maps
- Risk-weighted path cost vs. deterministic baseline
- Calibration: does CVaR_0.95 capture true 95th percentile?

What Is Missing

MLP stub produces uninformative samples. U-Net trained on occupancy data required. No comparison with deterministic inflation. No sample diversity or calibration evaluation.

⑦ Reviewer Critique

Weaknesses

- MLP stub cannot model spatial correlations — U-Net required
- Without training, all samples are random noise
- Diffusion inference latency may be too high for real-time (10 samples × 50 steps × U-Net)
- N=10 samples may produce high-variance CVaR estimates

Hidden Assumptions

- Training data captures deployment distribution
- MLP stub replaceable by U-Net without architectural changes
- N=10 sufficient for CVaR estimation

Failure Scenarios

- ☐ Score network not trained: all samples are random noise (module non-functional without training)
- Very dynamic environments: DDPM assumes temporal continuity
- Conditioning on stale history: prediction meaningless if observations are old

⑧ How to Turn This Into a Paper

Paper Title	Diffusion-Based Probabilistic Occupancy Prediction for CVaR-Optimal Robot Navigation
Venue	CoRL / ICRA — requires trained U-Net score network

Missing Experiments

- Train U-Net on robot occupancy data (ScanNet, nuScenes or TurtleBot3 custom)
- Compare CVaR map quality: diffusion vs. deterministic inflation vs. Gaussian belief
- Evaluate: does diffusion-based risk planning reduce collision rate in dynamic environments?
- DDIM acceleration: fewer steps for faster inference

Ablation Studies Needed

- MLP vs. U-Net score network: how much does architecture matter?
- N samples: 5 vs. 10 vs. 20 — CVaR accuracy vs. compute
- Cosine vs. linear noise schedule
- Conditioned vs. unconditioned diffusion

⑨ Open Research Questions

44. Can a consistency regularizer improve temporal coherence of generated maps?
45. Minimum N for accurate CVaR — theoretical lower bound?
46. Can diffusion model be trained to directly minimize navigation CVaR (differentiable planning)?
47. How does diffusion-based risk compare to particle filter in accuracy and compute?

⑩ Two-Minute Explanation for Anyone

Standard navigation systems make one prediction: 'this area is probably clear'. This module generates ten possible futures instead of one, using generative AI (diffusion models — same technology as image generation AI). From these ten futures, the worst-case scenarios are extracted and used for planning. The robot navigates safely even in the bad futures, not just the average. The technical framework is correctly implemented; the neural network inside needs to be trained on occupancy data before it produces meaningful predictions.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	8/10	Full maths + code
Publishability	7/10	Needs experiments
PhD Potential	8/10	Strong chapter candidate

Contribution 13 Latent World Model (Dreamer-v3 RSSM)

Category	Model-Based RL / Planning
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Reactive planners respond to current observations but cannot simulate consequences several steps ahead. A world model enables the robot to run hypothetical action sequences in an internal compressed space — 'imagination' — and select actions based on predicted outcomes, detecting irreversible situations before they occur.

Why does this problem exist?

Without forward simulation, a robot can only react to what it currently observes. Many failures (dead ends, irreversible commitments) are only detectable several steps ahead. Mental rollouts provide this lookahead.

Why is it hard?

Training a world model requires large amounts of navigation data and a powerful recurrent network. The model must capture both deterministic dynamics and stochastic uncertainty. The DreamerV3 challenge is computationally demanding.

② Intuition-First Explanation

Chess players think several moves ahead — they run sequences of hypothetical moves in their head before committing. A world model gives the robot the same ability: simulate a sequence of actions in a compressed internal space, evaluate their outcomes, pick the best one, then physically execute only the first action.

Analogy: A pilot practices in a flight simulator before the real flight. They try dangerous maneuvers in imagination without risk. The RSSM world model is that simulator — compact, fast, running entirely in the robot's processor — allowing safe mental practice of navigation strategies.

③ What I Built — Step by Step

- RSSM: $h_t = \text{GRU}(h_{t-1}, z_{t-1}, a_{t-1})$ — deterministic recurrent memory
- Prior: $z_t \sim p_\phi(z|h_t) = \mathcal{N}(\mu_\phi(h_t), \text{diag}(\sigma_\phi(h_t)^2))$ — imagined latent
- Posterior: $z_t \sim q_\phi(z|h_t, o_t)$ — observation-updated latent
- Reward predictor: $\hat{r} = g_\phi(h_t, z_t)$
- Mental rollout: H prior steps, no real observations
- Irreversibility penalty: subtract λ_{irrev} if final state non-returnable (C04)
- Action selection: $\text{argmax}_k G(\text{sequence}_k)$ via random shooting
- □ All networks are numpy MLP stubs — not trained

Data Flow

Current observation $\rightarrow$ posterior $(h_t, z_t) \rightarrow$ mental rollout H steps (prior only) $\times K$ candidates
 $\rightarrow$ score by $G = \sum \gamma^k \hat{r}_k - \text{irrev penalty} \rightarrow$ best first action

④ Deep Technical Breakdown

Key Components

- GRU-like transition: $h_t = \text{GRU}(h_{t-1}, z_{t-1}, a_{t-1})$ — deterministic path
- Prior network: $z_t \sim N(\mu_\phi(h_t), \sigma_\phi(h_t))$ — imagined future
- Posterior network: $z_t \sim N(\mu_q(h_t, o_t), \sigma_q)$ — real-observation update
- Reward predictor: $\hat{r} = g_\phi(h_t, z_t)$
- Random shooter: generates K candidate sequences, scores by G

Pseudocode / Algorithm

```
BELIEF UPDATE (real step):
    h_t = GRU(h_{t-1}, z_{t-1}, a_{t-1})
    z_t ~ posterior(h_t, o_t)          // incorporate real observation

MENTAL ROLLOUT (imagination):
    h, z = h_t, z_t                  // start from current belief
    G = 0
    for k in range(H):
        h = GRU(h, z, a_k)
        z ~ prior(h)                // imagined next latent
        G += gamma**k * reward_pred(h, z)
    if not returnable(h, z): G -= lambda_irrev
    return G

ACTION SELECTION:
    sequences = [random_sequence(H) for _ in range(K)]
    a_star = sequences[argmax(rollout(s) for s in sequences)]
    execute(a_star[0])              // only first action
```

Mathematical Formulation

RSSM: $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}), \quad z_t \sim p_\phi(z|h_t) \text{ or } q_\phi(z|h_t, o_t)$
Rollout value: $G(a) = \sum_{k=1}^H \gamma^{k-1} \cdot \hat{r}_k - \lambda \cdot 1[\neg \text{Ret}(h_H, z_H)]$
ELBO: $E_q[\sum_t \log p(o_t|h_t, z_t) + \log p(r_t|h_t, z_t) - \beta \cdot \text{KL}(q_\phi||p_\phi)]$
KL: $\text{KL}(N(\mu_q, \sigma_q) || N(\mu_p, \sigma_p)) = \log(\sigma_p/\sigma_q) + (\sigma_q^2 + (\mu_q - \mu_p)^2) / (2\sigma_p^2) - 1/2$

Complexity & Assumptions

Time complexity: Rollout: $O(H \cdot K \cdot |\text{RSSM}|)$. For $H=12, K=16$: 192 RSSM calls. Fast with small stubs; slow with full PyTorch RSSM.

- RSSM accurately models navigation dynamics $\square$ not trained
- Random shooting finds near-optimal sequences
- $H=12$ sufficient to detect irreversible situations
- Reward predictor signal informative before training

⑤ Research Meaning

Research Question

Can RSSM mental rollouts, augmented with returnability penalties, reduce irreversible state entry rate compared to reactive planning, and what is the minimum training data required?

Novelty

Integration of RSSM mental rollouts with returnability constraints (C04) as planning-time penalty is novel. Most DreamerV3 applications are game-playing; the adaptation to safe navigation with irreversibility penalties is underexplored.

Related Work

Hafner et al. (2019) DreamerV1, (2020) DreamerV2, (2023) DreamerV3. Ha & Schmidhuber (2018) World Models. Chua et al. (2018) PETS.

⑥ Results & Evidence

RSSM architecture executes correctly. Rollout returns discounted sums. Action selection produces non-trivial selections. □ Random weights — latent dynamics meaningless without training.

Metrics Used

- Imagined return G distribution across K sequences
- Irreversibility penalty application rate
- Action consistency (same state → same best action?)
- Comparison to reactive planning baseline

What Is Missing

Networks not trained. Random shooting suboptimal. No comparison with reactive planning in actual experiments.

⑦ Reviewer Critique

Weaknesses

- Random RSSM weights → randomly selected actions score highest by chance
- Random shooting highly suboptimal vs. CEM or MPC
- GRU stub very simplified vs. proper RSSM cell
- Without training: no meaningful world model exists

Hidden Assumptions

- RSSM trainable to accurate navigation dynamics from data
- H=12 sufficient for complex environments
- Reward predictor informative before training

Failure Scenarios

- □ Without training: all contributions theoretical
- Long corridors requiring $H > 12$ steps
- Highly stochastic environments where prior variance dominates

⑧ How to Turn This Into a Paper

Paper Title	Safety-Aware World Model Planning: RSSM Mental Rollouts with Returnability-Penalized Imagination
Venue	CoRL / NeurIPS Offline RL Workshop — after PyTorch training

Missing Experiments

- Implement PyTorch RSSM with proper backpropagation
- Train on 10K+ TurtleBot3 navigation episodes
- Compare to reactive A* baseline: does mental rollout reduce irreversible entries?
- CEM vs. random shooting for action sequence optimization

Ablation Studies Needed

- Trained RSSM vs. stub — does training matter? (Critical result)
- With vs. without irreversibility penalty
- H=6 vs. 12 vs. 24 — horizon sensitivity
- Random shooting vs. CEM

9 Open Research Questions

48. Can RSSM be fine-tuned online in new environments?
49. Can returnability check be differentiable, enabling gradient-based sequence optimization?
50. How does model error compound over rollout horizon?
51. Can hierarchical world model (semantic + metric) improve planning horizon?

10 Two-Minute Explanation for Anyone

Before moving, the robot mentally simulates what would happen if it tried different action sequences — without physically moving. This 'mental simulator' is a compact mathematical model of environment dynamics. The robot runs K hypothetical sequences through it, penalizes sequences that lead to irreversible situations, and executes the first action of the best sequence. The key challenge: the simulator must be trained on real navigation data to be accurate. Currently, the architecture is correctly implemented; training awaits.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	8/10	Full maths + code
Publishability	6/10	Needs experiments
PhD Potential	9/10	Strong chapter candidate

Contribution 14 Causal Risk Attribution (SCM + Do-Calculus)

Category	Causal AI / Failure Analysis
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

After a navigation failure, you need to know WHY it happened — not just that it happened. Was it sensor noise? A map error? Bad planning? Causal inference uses Pearl's do-calculus to identify the actual cause, not just variables that co-occurred.

Why does this problem exist?

Correlation analysis finds variables that appear together with failures, but cannot establish causation. Poor lighting might cause both high sensor noise AND bad obstacle detection independently. Causal inference traces the actual mechanism.

Why is it hard?

Correctly specifying the causal graph requires domain expertise. Structural equations must faithfully model real mechanisms. Counterfactual computation requires Monte Carlo averaging. Unobserved confounders can invalidate causal conclusions.

② Intuition-First Explanation

A doctor does not say 'fever and infection co-occur'. They reason about cause and effect: 'the infection caused the fever'. And they ask counterfactuals: 'If we had treated earlier, would the fever have developed?' This module enables the robot to ask the same about its own failures.

Analogy: An airline accident investigator traces the causal chain: iced pitot tube → false airspeed reading → autopilot error → crash. Each link is a causal relationship, not just a correlation. The investigator identifies which link to fix. This module is that investigator for robot navigation failures.

③ What I Built — Step by Step

- SCM with 6 navigation variables: sensor_noise → localization_error → obstacle_detection → path_risk → collision (+ map_accuracy branch)
- Structural equations: $V_i = f_i(PA_i, U_i)$ — linear with configurable coefficients
- Observational query: forward pass through causal graph given noise realizations
- Counterfactual: do($X=x$) — force X , keep same noise, recompute downstream
- ACE via Monte Carlo: $E[Y|do(X=x)] - E[Y|do(X=x')]$ over N noise samples
- Root-cause ranking: ablate each variable (set to 0), measure collision change
- 4 tests passing — all causal operations validated

Data Flow

Observed failure + noise estimate → SCM forward → counterfactual queries → ACE estimation
→ root-cause ranking → actionable diagnosis

④ Deep Technical Breakdown

Key Components

- SCM graph: 6 nodes, directed edges encoding causal relationships
- Structural equations: linear functions (hand-crafted — not learned from data)
- Counterfactual computer: mutilated model M_x with forced intervention
- ACE estimator: Monte Carlo over N noise samples
- Shapley approximator: ablation-based root-cause ranking

Pseudocode / Algorithm

```
OBSERVATIONAL QUERY:
vals = {}
for node in topological_order:    // ensures parents computed before children
    vals[node] = f_node(parent_vals, noise[node])

COUNTERFACTUAL do(X=x):
    for node in topological_order:
        if node == X: vals[node] = x    // forced intervention
        else:         vals[node] = f_node(parent_vals, noise[node])    // same noise

ACE ESTIMATION:
diffs = []
for _ in range(N):
    noise = {n: randn() for n in nodes}
    y_treat = counterfactual(noise, {X: x})[Y]
    y_ctrl  = counterfactual(noise, {X: x_prime})[Y]
    diffs.append(y_treat - y_ctrl)
return mean(diffs)
```

Mathematical Formulation

```
SCM:  $V_i = f_i(PA_i, U_i)$ ,  $PA_i$  = parents of  $V_i$ ,  $U_i$  = exogenous noise
Do-operator:  $P(Y|do(X=x)) = E_{\{PA_X\}}[P(Y|X=x, PA_X=pa)]$  (adjustment formula)
Counterfactual:  $Y_x(u)$  = solution of  $M_x$  with  $U = u$ 
ACE:  $E_U[Y_x(U)] - E_U[Y_{x'}(U)]$  estimated by Monte Carlo
Shapley approx:  $\phi_i = E_U[Y_{obs}(U) - Y_{\{V_i=0\}}(U)]$ 
```

Complexity & Assumptions

Time complexity: $O(N \cdot |V|)$ for ACE Monte Carlo. $O(N \cdot |V|^2)$ for all ablations. Tractable for small graphs.

- Causal graph correctly specified (hardest assumption)
- Structural equations faithfully model causal mechanisms
- Exogenous noises U_i are independent across nodes
- Linear equations adequate (may miss nonlinear interactions)

⑤ Research Meaning

Research Question

Can SCM do-calculus counterfactual reasoning identify actionable root causes of navigation failures that differ from correlational analysis, enabling targeted rather than blanket remediation?

Novelty

Application of SCM + counterfactual reasoning to navigation failure attribution is relatively novel in mobile robotics. Most safety analysis uses statistical methods. The coupling with Failure Explainer (C20) creates a unique diagnostic pipeline.

Related Work

Pearl (2009) Causality. Spirtes et al. (2000) — causal structure learning. Richens et al. (2020) — counterfactual diagnosis. Scholkopf et al. (2021) — causal representation learning.

⑥ Results & Evidence

Causal attribution produces interpretable rankings. ACE estimates converge with N. Counterfactual queries respond correctly to interventions. 4 tests passing.

Metrics Used

- ACE magnitude per variable
- Root cause ranking (sorted by $|\phi_i|$)
- Sensitivity to structural equation specification
- Comparison with correlational baseline

What Is Missing

Structural equations hand-crafted, not learned. Causal graph assumed, not discovered. No comparison with correlational analysis showing causal reasoning adds value.

⑦ Reviewer Critique

Weaknesses

- Hand-crafted linear structural equations may not model real failure mechanisms
- Causal graph may have wrong edges or wrong directions
- No identifiability analysis: when can causal effects be identified from observational data?
- No ground truth for causal attribution — validation is indirect

Hidden Assumptions

- Causal graph correctly specified
- Exogenous noises independent
- Linear equations adequate

Failure Scenarios

- Incorrectly specified graph: attribution to non-causal variable
- Feedback loops in real systems (not modeled)
- Unobserved confounders not in graph

⑧ How to Turn This Into a Paper

Paper Title	Causal Attribution of Robot Navigation Failures via Structural Causal Models and Counterfactual Reasoning
Venue	ICRA / NeurIPS Causal ML Workshop / UAI

Missing Experiments

- Learn structural equations from navigation episode data (causal structure learning)
- Compare causal vs. correlational ranking on same failure data
- Human evaluation: do engineers find causal attributions more actionable?
- Ground-truth study: inject known failures, evaluate attribution accuracy

Ablation Studies Needed

- Learned vs. hand-crafted structural equations
- Linear vs. nonlinear structural equations
- N samples for ACE convergence analysis
- Full causal graph vs. partial — how much does missing context matter?

9 Open Research Questions

52. Can causal graph be learned from navigation episodes (PC, FCI algorithms)?
53. Can causal attribution guide automated remediation?
54. Can a causal model be used for counterfactual data augmentation?
55. How do causal relationships change across environment types?

10 Two-Minute Explanation for Anyone

When something goes wrong, you need the real reason — not just what was present when the crash happened. This module builds a mathematical cause-and-effect graph of the navigation system: sensor noise causes positioning error, which causes incorrect obstacle detection, which causes bad planning, which causes collisions. After a failure, it asks: 'If sensor noise had been zero, would the crash still have happened?' This counterfactual question — enabled by Pearl's do-calculus — identifies the actual root cause. The result is actionable: fix the identified cause, not everything at once.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	8/10	Full maths + code
Publishability	7/10	Needs experiments
PhD Potential	9/10	Strong chapter candidate

Contribution 15 Neuromorphic Sensing (DVS + LIF-SNN)

Category	Neuromorphic / Bio-Inspired Perception
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Frame-based cameras capture images 30 times per second — 33ms latency. For fast-moving robots, this is too slow. Event cameras (DVS) fire asynchronously per-pixel at microsecond resolution when brightness changes. This module simulates a DVS camera and processes its output with a Spiking Neural Network.

Why does this problem exist?

High-speed navigation requires latencies far below 33ms. DVS cameras provide microsecond-precision motion detection with orders-of-magnitude lower power and bandwidth. SNNs match this event-driven paradigm.

Why is it hard?

Processing asynchronous event streams requires non-standard algorithms — standard CNNs cannot directly process events. Time surfaces and SNNs are required. SNN training is challenging (surrogate gradients needed).

② Intuition-First Explanation

When watching a 200km/h tennis ball on TV, it blurs. Professional players see it clearly because the human visual system has specialized motion detection that does not wait for 'frames'. DVS cameras replicate this biological property with microsecond precision.

Analogy: Smoke detectors do nothing until they detect a change. A photoframe that updates every 30 seconds would miss a fire starting between updates. The DVS camera is the smoke detector — firing instantly on any change — while standard cameras are the slow-updating photoframe.

③ What I Built — Step by Step

- DVS simulator: $\Delta L = \log(f_2) - \log(f_1)$, fire ON if $\Delta L \geq C_{\text{pos}}$, OFF if $\Delta L \leq -C_{\text{neg}}$
- Refractory period: min time between events per pixel
- Gaussian noise events at rate p (realism model)
- Time surface: $T_p(x,y,t) = \exp(-(t - t_{\text{last}_p(x,y)})/\tau)$ — 2-channel representation
- LIF neuron: $V_{t+1} = e^{-\Delta t/\tau_m} V_t + (1 - e^{-\Delta t/\tau_m}) I_t$, spike if $V \geq V_{\text{thresh}}$
- SNN grid: spatial grid of LIF neurons, each receiving weighted time surface input
- Output: obstacle probability = spike_count / N_steps per grid cell
- ☐ SNN weights are random — no training performed

Data Flow

Frame pair $\rightarrow$ DVS events $\rightarrow$ time surface (2-channel) $\rightarrow$ SNN LIF neurons (N steps) $\rightarrow$ spike counts $\rightarrow$ obstacle probability grid

④ Deep Technical Breakdown

Key Components

- DVS simulator: per-pixel log-intensity threshold comparison
- Noise model: Gaussian noise events at configurable rate
- Time surface builder: exponential decay T_p per polarity channel
- LIF neuron: correct discrete dynamics with refractory period
- SNN grid: spatial layout, random weights (awaiting training)

Pseudocode / Algorithm

```
DVS SIMULATION:
for each pixel (x, y):
    delta = log(frame2[y,x]+1e-6) - log(frame1[y,x]+1e-6)
    if delta >= C_pos and refractory_ok(x,y):
        events.append(Event(x, y, t, polarity=+1))
    elif delta <= -C_neg and refractory_ok(x,y):
        events.append(Event(x, y, t, polarity=-1))

TIME SURFACE:
for each event e:  $T_p(e.x, e.y) = \max(T_p(e.x, e.y), \exp(-(t-e.t)/\tau))$ 

LIF STEP:
 $V = \exp(-dt/\tau_m) * V + (1 - \exp(-dt/\tau_m)) * I_{ext}$ 
if  $V \geq V_{thresh}$ : spike;  $V = V_{reset}$ 
```

Mathematical Formulation

```
DVS event:  $e=(x,y,t,p)$  iff  $|\Delta L(x,y,t)| \geq C_p$  where  $\Delta L = \log I(t) - \log I(t_{prev})$ 
Time surface:  $T_p(x,y,t) = \exp(-(t - t_p^{last}(x,y))/\tau)$ 
LIF continuous:  $\tau_m \frac{dV}{dt} = -V + I_{ext}$ 
LIF discrete:  $V_{t+1} = e^{-\Delta t/\tau_m} \cdot V_t + (1 - e^{-\Delta t/\tau_m}) \cdot I_t$ 
Output:  $\hat{O}_{ij} = \text{spike\_count}_{ij} / N_{steps} \in [0,1]$ 
```

Complexity & Assumptions

Time complexity: DVS: $O(W \cdot H)$ per frame. Time surface: $O(N_{events})$. SNN: $O(\text{grid}_n \cdot \text{grid}_m \cdot N_{steps})$. All fast.

- LIF adequate for spike-based processing (ignores higher-order neuron dynamics)
- Time surface sufficient to represent event stream information
- ☐ Random SNN weights — output meaningless without training
- DVS simulator faithfully replicates real DVS camera behavior

⑤ Research Meaning

Research Question

Can DVS event output processed by a LIF-SNN provide spatial obstacle probability maps with lower detection latency than frame-based approaches in high-speed navigation scenarios?

Novelty

Full data pipeline: DVS simulation → time surface → SNN, integrated with DynNav navigation stack. Most DVS robotics papers are standalone; this integration is the differentiator.

Related Work

Gallego et al. (2020) — event-based vision survey. Zhu et al. (2018) — event-based optical flow. Gehrig et al. (2020) — end-to-end learning. Orchard & Bhattacharyya — SNN for robotics.

⑥ Results & Evidence

DVS simulator validated. LIF neurons correct. Time surface correct. ☐ Random SNN weights → random obstacle detection.

Metrics Used

- Detection latency (event → obstacle flag)
- Spatial accuracy: spike rate vs. obstacle location
- Sensitivity: minimum detectable obstacle speed
- False positive rate from noise events

What Is Missing

SNN not trained. No comparison with frame-based detection. No real DVS hardware evaluation.

⑦ Reviewer Critique

Weaknesses

- ☐ Without SNN training, output is random — module cannot perform obstacle detection
- Single-layer SNN — deep or convolutional SNN needed for complex patterns
- Time surface discards temporal ordering within window
- Constant threshold C across all pixels (real DVS has per-pixel variation)

Hidden Assumptions

- Constant threshold C across all conditions
- Time surface with fixed τ sufficient for all velocity ranges
- LIF adequate for required spatial processing

Failure Scenarios

- Very slow objects below DVS threshold (no events generated)
- High-noise environments overwhelming real motion events
- Static obstacles: DVS only detects change (invisible unless camera moves)

⑧ How to Turn This Into a Paper

Paper Title	Event-Driven Obstacle Detection for Robot Navigation: DVS Simulation and LIF-SNN Spatial Processing Pipeline
Venue	CVPR Neuromorphic Workshop / IEEE Sensors Journal

Missing Experiments

- Train SNN on labeled DVS obstacle dataset (N-MNIST, N-Caltech or custom)

- Compare latency: DVS+SNN vs. frame-based CNN on high-speed scenarios
- Real DVS camera evaluation on TurtleBot3
- Comparison with Asynet (event-based graph neural net)

Ablation Studies Needed

- Time surface τ sensitivity
- SNN depth: 1-layer vs. conv-SNN
- DVS threshold C: detection rate vs. event rate
- With vs. without refractory period

⑨Open Research Questions

56. Can SNN be trained end-to-end on navigation episodes?
57. What is the minimum achievable latency (DVS+SNN vs. frame-based theoretical bound)?
58. Can DVS fuse with LiDAR for complementary sensing?
59. Can online-learning SNN adapt to new obstacle types?

⑩Two-Minute Explanation for Anyone

Normal cameras take pictures 30 times per second — there's at least 33ms of delay before a moving obstacle is detected. Event cameras work differently: each pixel fires independently the instant it detects any brightness change, with microsecond precision. This module simulates this type of camera and processes its output with a biologically-inspired Spiking Neural Network. The full pipeline architecture is implemented and validated; the neural network awaits training on event-based data to produce meaningful obstacle detection.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	8/10	Strong chapter candidate

Contribution 16 Federated Navigation Learning (FedAvg + DP)

Category	Federated Learning / Privacy
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Six robots work in six different buildings. Each accumulates navigation knowledge, but their maps are private. Federated learning enables all robots to benefit from each other's experience without anyone sharing their private sensor data.

Why does this problem exist?

Data privacy is increasingly important in sensitive environments (hospitals, homes, corporate facilities). Raw maps constitute private spatial information. Federated learning enables collective improvement while respecting privacy.

Why is it hard?

Federated learning under data heterogeneity (robots in very different environments have different distributions) is a known challenge. Differential privacy adds noise that degrades model quality. Byzantine clients can poison the shared model.

② Intuition-First Explanation

Your phone's keyboard improves autocomplete without sending your messages to a server. Each phone trains locally, sends only tiny model updates, and receives an improved global model. This is federated learning — and it is what this module implements for robot navigation knowledge sharing.

Analogy: A group of chefs from different restaurants want to collectively improve their cooking. They cannot share secret recipes, so they each distill their learning into abstract 'cooking principles' and share only those. The principles are combined into a shared guidebook everyone benefits from.

③ What I Built — Step by Step

- FedAvg server: $w^{t+1} = \sum_k (n_k/n) w_k^t$
- Robot client: E epochs local SGD, return updated weights
- DP mechanism: clip to C, add $N(0, \sigma^2 I)$, $\sigma = C\sqrt{(2\ln(1.25/\delta))/\epsilon}$
- Two aggregation modes: weighted (by n_k) and uniform
- Validation: val_MSE tracked per round on synthetic holdout
- ☐ Clients train on synthetic random data — not real navigation data

Data Flow

Global model → broadcast → local training (E epochs) → DP noise → delta w_k → FedAvg aggregate → new global model

④ Deep Technical Breakdown

Key Components

- FedAvg aggregator: weighted average by training set sizes
- Local SGD trainer: E epochs on private data
- Gradient clipper: $\min(1, C/\|w\|) \cdot w$
- Gaussian mechanism: $+ N(0, \sigma^2 I)$
- Validation evaluator: test on synthetic holdout per round

Pseudocode / Algorithm

```
SERVER:
w_global = random_init()
for round in range(T):
    [broadcast w_global to all clients]
    updates = [client_k.train(w_global) for k in range(K)]
    w_global = sum(n_k/n * w_k for (w_k, n_k) in updates)

CLIENT k:
w = copy(w_global)
for epoch in range(E):           // local SGD
    for (x, y) in local_data: update(w)
w_clipped = w * min(1, C/norm(w)) // clip
if dp: w += randn(shape) * sigma   // DP noise
return (w_clipped, len(local_data))
```

Mathematical Formulation

```
FedAvg:  $w^{t+1} = \sum_k (n_k/n) w_k^t$ 
Global objective:  $F(w) = \sum_k (n_k/n) L_k(w)$ 
DP sensitivity:  $\Delta_2(f) \leq C$  (bounded by clipping)
Gaussian mechanism:  $\sigma = C \cdot \sqrt{(2 \ln(1.25/\delta)) / \epsilon} \rightarrow (\epsilon, \delta)$ -DP
Communication per round:  $O(|w| \cdot K)$ 
```

Complexity & Assumptions

Time complexity: Local training: $O(E \cdot n_k \cdot |w|)$. Aggregation: $O(K \cdot |w|)$. Communication: $O(K \cdot |w|)$ per round.

- Data distribution i.i.d. across clients (strongly violated in heterogeneous fleet)
- Clients honest (Byzantine not handled — see C26)
- Linear model w adequate for navigation policy
- ☐ Synthetic data distribution matches real navigation data

⑤ Research Meaning

Research Question

Can federated learning with (ϵ, δ) -DP achieve comparable navigation policy quality to centralized training, while providing formal privacy guarantees about individual robots' spatial data?

Novelty

FL with DP for robot navigation policy sharing is underexplored. Most FL work is for image classification. Adapting to navigation (policy, not classifier) introduces challenges with reward vs. supervised learning.

Related Work

McMahan et al. (2017) FedAvg. Dwork & Roth (2014) DP foundations. Li et al. (2020) FL survey.

⑥ Results & Evidence

Federated convergence: val_MSE decreases over 20 rounds. DP noise effect quantified. □ Synthetic data only — navigation performance not measured.

Metrics Used

- Val MSE per round
- Communication rounds to convergence
- DP noise impact: accuracy under (ϵ, δ) -DP vs. non-private
- Heterogeneity impact on convergence

What Is Missing

No real navigation data evaluation. Privacy guarantee not compared against actual leakage risk. Non-i.i.d. heterogeneity not addressed.

⑦ Reviewer Critique

Weaknesses

- FedAvg assumes i.i.d. data — violated in heterogeneous fleet
- Synthetic training: val_MSE convergence disconnected from navigation performance
- Linear model too simple for navigation policy learning
- Byzantine clients not handled

Hidden Assumptions

- i.i.d. data across clients
- Clients honest
- Linear model sufficient

Failure Scenarios

- Non-i.i.d. data: divergent gradient directions → FedAvg diverges
- Byzantine client: gradient poisoning corrupts global model
- Communication bottleneck: large navigation policy models

⑧ How to Turn This Into a Paper

Paper Title	Privacy-Preserving Federated Navigation Policy Learning with Differential Privacy Guarantees
Venue	ICRA / AAMAS / FL-ICML Workshop

Missing Experiments

- Train on real navigation data: TurtleBot3 episodes in different buildings
- Evaluate: federated policy collision rate vs. centralized
- Non-i.i.d. analysis: heterogeneous environments impact on convergence

- Local-only vs. federated: does federation add value?

Ablation Studies Needed

- With DP vs. without DP: privacy cost in navigation performance
- FedAvg vs. FedProx (handles heterogeneity better)
- K=2 vs. 4 vs. 6 vs. 10 robots
- Linear vs. MLP vs. CNN navigation policy

⑨Open Research Questions

- 60. Can Byzantine-robust aggregation (C26 median) apply to gradient aggregation?
- 61. What ϵ is sufficient for robot navigation data privacy?
- 62. Can FL combine with curriculum learning (C22) across heterogeneous fleet?
- 63. Can robots negotiate communication frequency by bandwidth?

⑩Two-Minute Explanation for Anyone

Six robots in six different buildings each learn navigation tricks. They want to share knowledge, but their maps are private. Federated learning solves this: each robot trains its local model, adds mathematical noise to hide what data it used, and sends only the noisy model update to a server. The server combines all updates into a better shared model that everyone benefits from. The mathematical privacy guarantee ensures the shared model cannot reveal what any specific robot observed.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	8/10	Full maths + code
Publishability	7/10	Needs experiments
PhD Potential	8/10	Strong chapter candidate

Contribution 17 Topological Semantic Maps (CLIP Grounding)

Category	Semantic Mapping / Language Navigation
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Dense metric maps (every cm^2) are expensive and do not support natural language. 'Go to the kitchen' cannot be looked up in an occupancy grid. Topological maps represent space as a graph of named places connected by traversable paths, enabling natural language navigation at low computational cost.

Why does this problem exist?

Human navigation is fundamentally semantic: 'meeting room on floor 2'. Metric coordinates are unnatural for non-experts. A representation supporting both efficient navigation and language understanding bridges human-robot communication.

Why is it hard?

Connecting text labels to visual places requires a cross-modal embedding model (CLIP). The graph must be built incrementally and support dynamic edge invalidation. CLIP embeddings require a live model — the stub implementation cannot perform real open-vocabulary grounding.

② Intuition-First Explanation

A hotel directory lists rooms by name (Room 301, Gym, Restaurant) connected by corridors and elevators. This is a topological map. When you ask 'where is the gym?', you look up the name, find the node, follow the path. CLIP extends this so 'workout area' finds 'gym' even if the exact label differs.

Analogy: Google Maps shows named landmarks and routes between them — not centimeter-level terrain maps. Route planning on the landmark graph is fast and intuitive. This module implements the equivalent for indoor robot navigation with language understanding.

③ What I Built — Step by Step

- TopologicalSemanticMap: graph $G = (V, E, w)$ of semantic zones
- Each node: label, centroid (x, y) , CLIP embedding $e_i \in \mathbb{R}^{512}$
- `add_node()`, `add_edge()`, `invalidate_edge()` for dynamic building
- Dijkstra planner on the graph
- Open-vocabulary grounding: $\arg\max_i \cos(e_i, \text{CLIP.encode(query)})$
- Stub CLIP: deterministic hash $\square$ (real CLIP not connected)
- Levenshtein fallback for fuzzy label matching
- JSON serialization for persistence across sessions
- 4 tests passing including round-trip JSON

Data Flow

Text query → CLIP embed → cosine similarity over all nodes → target node → Dijkstra path → metric centroid waypoints → A for within-zone navigation*

④ Deep Technical Breakdown

Key Components

- Graph structure: adjacency list with passable flag per edge
- CLIP embedder: stub (hash) → needs real CLIP for open-vocabulary
- Dijkstra: $O(|V|^2 \text{ or } |E|\log|V|)$ shortest path
- Edge invalidator: marks edges impassable on obstacle discovery
- JSON serializer: persistent cross-session topological map

Pseudocode / Algorithm

```
ADD NODE AND EDGE:
    k_id = map.add_node('kitchen', centroid=(2.0,3.0), emb=CLIP('kitchen'))
    c_id = map.add_node('corridor', centroid=(5.0,1.0), emb=CLIP('corridor'))
    map.add_edge(k_id, c_id, weight=3.2, bidirectional=True)

OPEN-VOCABULARY GROUNDING:
    q_emb = CLIP.encode(user_query)    // e.g. 'go to the cafeteria'
    v_star = argmax over nodes: cosine(node.embedding, q_emb)
    OR: argmin over nodes: levenshtein(node.label, user_query) // fallback

PLAN AND NAVIGATE:
    path, cost = map.plan(start_id, v_star) // Dijkstra
    waypoints = [map.nodes[id].centroid for id in path]
    navigate_metric(waypoints)             // A* within zones
```

Mathematical Formulation

```
Graph:  $G = (V, E, w)$  where  $w_{ij}$  = travel cost between zone centroids
Dijkstra:  $d(v_s, v_g) = \min \text{ path weight}$ 
Cosine similarity:  $\cos(a,b) = a \cdot b / (||a|| \cdot ||b||)$ 
CLIP grounding:  $v^* = \operatorname{argmax}_i \cos(\text{CLIP}(\text{label}_i), \text{CLIP}(\text{query}))$ 
Levenshtein:  $d(a,b) = \text{edit distance} - O(|a| \cdot |b|)$  per pair
```

Complexity & Assumptions

Time complexity: Dijkstra: $O(|V|^2 \text{ or } |E|\log|V|)$. CLIP grounding: $O(|V| \cdot d)$. Edge invalidation: $O(|E|)$. JSON: $O(|V| + |E|)$.

- Zones meaningfully distinct and non-overlapping
- CLIP embeddings correctly capture semantic similarity □ stub cannot do this
- Dijkstra path corresponds to navigable route
- Zone centroids are navigable locations

⑤ Research Meaning

Research Question

Does topological-semantic representation enable open-vocabulary zone navigation while reducing planning complexity compared to metric grid search?

Novelty

Integration of CLIP-based open-vocabulary grounding with dynamic topological map supporting edge invalidation and Dijkstra replanning in navigation context. Standard topological navigation uses pre-defined labels; CLIP enables arbitrary natural language.

Related Work

Kuipers et al. (2000) topological maps. Shafiullah et al. (2022) CLIP-Fields. LERF — language embedded radiance fields. Concept Graphs — open-vocab 3D scene graphs.

⑥ Results & Evidence

Dijkstra correct. Edge invalidation reroutes correctly. JSON round-trip verified. ☐ CLIP grounding not evaluated with real model.

Metrics Used

- Zone matching accuracy
- Path length vs. metric A* baseline
- Replanning success after invalidation
- CLIP grounding latency

What Is Missing

CLIP not connected. No open-vocabulary evaluation with real queries. No comparison with metric planning. No real-time map building evaluation.

⑦ Reviewer Critique

Weaknesses

- Stub CLIP provides correct results only for exact label matches
- Graph topology must be manually specified — no automatic zone detection
- Dijkstra ignores geometry within zones
- Zone centroids may not be most navigable point

Hidden Assumptions

- Zones have meaningful CLIP embeddings
- Graph consistent with physical connectivity
- Zone boundaries are clear

Failure Scenarios

- Ambiguous descriptions matching multiple zones
- Unknown zones not in graph
- Dynamic zone changes not propagated to graph

⑧ How to Turn This Into a Paper

Paper Title	Open-Vocabulary Semantic Zone Navigation via CLIP-Grounded Topological Maps with Dynamic Edge Invalidation
--------------------	--

Venue	ICRA / RA-L / CoRL
-------	--------------------

Missing Experiments

- Connect real CLIP: evaluate grounding on 50+ natural language queries
- Automatic zone detection from visual SLAM + semantic segmentation
- Real robot: TurtleBot3 navigating with verbal instructions
- Comparison with metric A* on same environments

Ablation Studies Needed

- Real CLIP vs. stub: grounding accuracy improvement
- Topological vs. metric A* on same environments
- With vs. without Levenshtein fallback
- Static vs. dynamic edge invalidation

⑨ Open Research Questions

64. Can zone boundaries be automatically detected using SAM + CLIP?
65. Can topological map be built collaboratively by multiple robots (C09)?
66. How to resolve conflicting instructions when multiple zones match?
67. Can map generalize across similar buildings?

⑩ Two-Minute Explanation for Anyone

Instead of centimeter-level grid maps, this module stores space as a graph of named places — 'kitchen', 'corridor', 'exit' — connected by edges. When given a language instruction, it finds the place whose text description is most similar to the instruction (using CLIP language embeddings), then plans a graph route. When obstacles block a connection, that edge is automatically removed. The system can be saved and reloaded across sessions. Real CLIP integration would enable full open-vocabulary grounding; currently a stub is used.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	7/10	Prototype level
Publishability	7/10	Needs experiments
PhD Potential	8/10	Strong chapter candidate

Contribution 18 Formal Safety Shields (STL + CBF) ★

Category	Formal Methods / Runtime Safety — STRONGEST CONTRIBUTION
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Neural network planners and RL agents can occasionally produce unsafe velocity commands — driving the robot into an obstacle. Formal safety shields provide a mathematical guarantee that NO unsafe command will ever be executed, regardless of what the upstream planner outputs.

Why does this problem exist?

'Probably safe' is insufficient for robots near humans. A 99.9% safe policy still fails 0.1% of the time. Formal methods provide provable safety — not statistical, but mathematical. This is the standard demanded for safety-critical systems.

Why is it hard?

CBF conditions are derived for continuous-time dynamics. Discrete-time implementation introduces approximation. Multiple simultaneous obstacles create conflicting constraints. STL temporal logic monitoring requires correct specification of time bounds.

② Intuition-First Explanation

A nuclear power plant's control software manages the reactor. But a separate, independent safety system monitors everything and automatically shuts down if ANY threshold is crossed — regardless of what the control software says. The STL monitor is that watchdog; the CBF filter is the emergency shutdown that modifies commands to stay safe.

Analogy: A building has smoke detectors (STL monitoring — detects violations) and a fire suppression system (CBF filtering — active intervention to prevent catastrophe). Neither alone is sufficient; together they provide defense in depth.

③ What I Built — Step by Step

- STL compositional formulas: atoms, $\Box[a,b]$ (always), $\Diamond[a,b]$ (eventually), $\wedge$, $\vee$
- STL robustness ρ : positive = spec satisfied, negative = violated, magnitude = margin
- STL monitor: accumulates state trajectory, evaluates ρ each step, logs violations
- CBF: $h(x) = \|\text{robot} - \text{obs}\| - r_{\text{safe}}$, safe set $C = \{h(x) \geq 0\}$
- CBF condition: $\nabla h(x) \cdot u + \alpha \cdot h(x) \geq 0 \rightarrow$ forward invariance of C
- Gradient projection QP: $u^* = \arg\min \|u - u_{\text{des}}\|^2$ s.t. CBF constraint
- SafetyShield: wraps any planner — one interface for both components
- Experiment: 50 episodes shielded vs. unshielded
- RESULTS: 4.2 $\rightarrow$ 0.3 violations/episode (93% reduction), <8% path overhead

Data Flow

Desired command $u_{des} \rightarrow$ CBF gradient projection $\rightarrow u_{safe}$ AND State trajectory $\rightarrow$ STL robustness $\rho \rightarrow$ violation log

④ Deep Technical Breakdown

Key Components

- STL atom: $\rho(\mu, x, t) = \text{predicate_function}(x[t])$
- STL $\Box[a,b]$: $\rho = \min_{t' \in [t+a, t+b]} \rho(\phi, x, t')$
- STL $\Diamond[a,b]$: $\rho = \max_{t' \in [t+a, t+b]} \rho(\phi, x, t')$
- CBF barrier: $h(x) = \|x - x_{obs}\| - r_{safe}$
- CBF gradient: $\nabla h = (x - x_{obs}) / \|x - x_{obs}\|$
- Gradient projection: iterative correction of u_{des}
- Shield interface: $\text{STL.update}(\text{state}) + \text{CBF.filter}(u, \text{pos}, \text{obstacles}) \rightarrow u_{safe}$

Pseudocode / Algorithm

```
STL ROBUSTNESS (recursive):
    rho_always(phi, x, t, a, b) = min(rho(phi, x, t') for t' in [t+a, t+b])
    rho_eventually(phi, x, t, a, b) = max(rho(phi, x, t') for t' in [t+a, t+b])
    rho_and(phi1, phi2, x, t) = min(rho(phi1, x, t), rho(phi2, x, t))

CBF GRADIENT PROJECTION:
    u = u_desired
    for obs_i in obstacles:
        h_i = norm(robot - obs_i) - r_safe
        grad_h = (robot - obs_i) / norm(robot - obs_i)
        for iteration in range(max_iter):
            lf_h = dot(grad_h, u)
            if lf_h + alpha * h_i >= 0: break // constraint satisfied
            u += step * max(0, -(lf_h + alpha*h_i)) * grad_h
    return u

SAFETY SHIELD STEP:
    rho = stl_monitor.update(state)
    u_safe, info = cbf_filter.filter(u_desired, robot_pos, obstacles)
    return u_safe, {'stl': rho, 'cbf': info}
```

Mathematical Formulation

```
STL always:  $\rho(\Box[a,b]\phi, x, t) = \min_{t' \in [t+a, t+b]} \rho(\phi, x, t')$ 
CBF condition:  $\nabla h(x) \cdot f(x, u) + \alpha \cdot h(x) \geq 0 \rightarrow d/dt h(x(t)) \geq -\alpha \cdot h(x(t))$ 
Forward invariance:  $h(x_0) \geq 0 \wedge \text{CBF cond.} \forall t \rightarrow h(x_t) \geq 0 \forall t > 0$ 
Min-norm QP:  $u^* = \text{argmin} \|u - u_{des}\|^2 \text{ s.t. } \nabla h(x) \cdot u \geq -\alpha \cdot h(x)$ 
Gradient projection:  $u^{k+1} = u^k - (\nabla h \cdot u^k + \alpha h) / \|\nabla h\|^2 \cdot \nabla h$  when violated
```

Complexity & Assumptions

Time complexity: STL: $O(T \cdot |\text{formula}|)$ for T history steps. CBF: $O(N_{obs} \cdot \text{max_iter})$ per step. Both real-time capable.

- Obstacles are static during CBF computation interval
- Gradient projection converges within max_iter (guaranteed for convex QP)

- CBF condition derived for continuous-time holds approximately in discrete-time
- STL bounds $[a,b]$ cover the relevant safety horizon

⑤ Research Meaning

Research Question

Can a combined STL-CBF safety shield reduce constraint violations by >80% with <10% efficiency overhead, for any upstream planner including adversarial or random policies?

Novelty

Combination of STL monitoring (temporal violation detection) with CBF command filtering (real-time constraint enforcement) in a unified navigation safety shield, experimentally validated with quantitative before/after comparison. Most formal safety work addresses only one component.

Related Work

Donze & Maler (2010) STL robustness. Ames et al. (2019) CBF theory. Raman et al. (2015) MPC+STL. Liu & Tomizuka (2014) CBF for safe learning.

⑥ Results & Evidence

★ STRONGEST RESULTS IN REPO: 4.2→0.3 violations/episode (93%), path overhead <8%, mean CBF correction 0.026 m/s. Quantitative, meaningful, reproducible.

Metrics Used

- Safety violations per episode
- Mean CBF correction norm (m/s)
- Path length overhead: shielded vs. unshielded
- STL robustness p distribution over time
- Violation reduction percentage

What Is Missing

CBF under uncertainty (stochastic obstacles) not analyzed. STL with nested temporal operators not stress-tested. No comparison with model-predictive safety filtering.

⑦ Reviewer Critique

Weaknesses

- Gradient projection is first-order QP approximation — may not find global minimum-perturbation
- CBF for continuous-time — discrete-time introduces approximation
- Multiple obstacles handled by independent CBF — may be overly conservative
- STL monitoring is retroactive — violations logged but not prevented by STL alone

Hidden Assumptions

- Obstacles static during CBF interval
- Gradient projection converges within `max_iter`
- CBF condition sufficient in discrete time

Failure Scenarios

- Very fast dynamics where CBF cannot correct fast enough
- Multiple simultaneous obstacles with conflicting CBF constraints
- Narrow passages where CBF safe set is empty

⑧ How to Turn This Into a Paper

Paper Title	★ Composable STL-CBF Runtime Safety Shields for Learning-Augmented Autonomous Navigation
Venue	★ ICRA 2026 / IEEE RA-L — publishable now with minor additions

Missing Experiments

- Formal proof: CBF gradient projection finds minimum-perturbation safe command
- Stochastic CBF: extension to uncertain obstacle positions
- Comparison with MPC-based safety filtering
- Hardware: TurtleBot3 with physical obstacles

Ablation Studies Needed

- CBF gradient projection vs. exact QP solver
- STL monitoring alone vs. CBF alone vs. combined
- Single obstacle CBF vs. multiple
- α (CBF gain) sensitivity: safety vs. efficiency

⑨ Open Research Questions

68. Can CBF α be automatically tuned to balance safety and efficiency?
69. Can STL specs be synthesized from natural language safety requirements?
70. Can CBF extend to stochastic dynamics with probabilistic guarantees?
71. Can the shield learn from past violations to predict and prevent future ones?

⑩ Two-Minute Explanation for Anyone

Even the best planner occasionally outputs a dangerous command. This module is the robot's safety belt: every velocity command passes through a mathematical filter before execution. The filter checks: 'Will this command keep the robot at a safe distance from all obstacles?' If not, it modifies the command as little as possible — with a mathematical proof that the modification is sufficient. Alongside this, a temporal logic monitor tracks whether safety specifications have been maintained over time. Result: 93% fewer violations, <8% efficiency cost. This is the most mathematically rigorous and experimentally validated contribution in the repository.

Evaluation Scores

Dimension	Score	Comment
Originality	9/10	Genuinely novel

Technical Depth	9/10	Full maths + code
Publishability	9/10	Conference-ready now
PhD Potential	9/10	Strong chapter candidate

Contribution 19 LLM Mission Planner

Category	NLP / Robot Instruction Interface
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Operators should say 'go to the cafeteria, then the storage room, return to base' instead of entering three sets of coordinates. This module parses natural language into ordered waypoint sequences using an LLM, with offline keyword fallback.

Why does this problem exist?

Natural language interfaces democratize robot operation. LLMs have world knowledge to understand spatial language without task-specific training. An offline fallback ensures robustness when the API is unavailable.

Why is it hard?

LLMs are slow (API latency), produce unstructured text by default, and can hallucinate. Structured JSON prompting, confidence gating, and keyword fallback are all required for safe deployment.

② Intuition-First Explanation

You speak to a receptionist: 'Please tell Dr. Kim I'm here'. The receptionist converts your instruction into a structured action: look up Dr. Kim, send a message to room 412. The LLM Mission Planner is that receptionist for the robot.

Analogy: Google Assistant converting 'set an alarm for 7am and remind me to take medicine' into two structured API calls. The LLM understands intent and generates structured output.

③ What I Built — Step by Step

- HTTP client for Ollama (local) and OpenAI-compatible API
- Structured prompt: request JSON {confidence, waypoints[{label,priority,action}]}
- JSON parser: strip markdown fences, validate, handle missing fields
- Confidence gating: reject if $c < \tau_c$
- Keyword fallback: scan for zone names in text, sort by position
- Levenshtein fuzzy match for near-match labels
- Metric resolution: label $\rightarrow$ (x,y) via zone_map dict
- 6 tests passing including fallback and fuzzy matching

Data Flow

Natural language instruction $\rightarrow$ LLM API (or keyword fallback) $\rightarrow$ JSON parse $\rightarrow$ Mission (ordered Waypoints) $\rightarrow$ metric coordinates $\rightarrow$ planner

④ Deep Technical Breakdown

Key Components

- LLM client: HTTP POST to Ollama/OpenAI endpoint
- JSON parser: robust to formatting variations
- Keyword extractor: zone name scanning + position sorting
- Levenshtein matcher: fuzzy label resolution
- Metric resolver: zone_name $\rightarrow$ (x,y) lookup

Pseudocode / Algorithm

```
LLM QUERY:
payload = {model: name, messages: [system_prompt, user_instruction]}
response = http_post(endpoint, payload, timeout=10)
goal = parse_json(response.message.content)
if goal is None or goal.confidence < tau_c:
    return keyword_fallback(instruction)

KEYWORD FALLBACK:
found = []
for zone in known_zones:
    if zone in instruction.lower():
        found.append((instruction.find(zone), zone))
found.sort() // by position in text  $\rightarrow$  priority order
return Mission([Waypoint(zone, priority=i) for i, (_, zone) in enumerate(found)])

FUZZY MATCH:
best = argmin over zones: levenshtein(zone, label)
```

Mathematical Formulation

```
LLM:  $P(W|x) = \prod_k p(w_k|w_{1:k-1}, x)$  – autoregressive generation
JSON schema: {confidence: [0,1], waypoints: [{label:str, priority:int, action:str}]}
Fallback: sort by string position index – heuristic priority order
Levenshtein:  $d(a,b)$  = edit distance –  $O(|a| \cdot |b|)$ 
```

Complexity & Assumptions

Time complexity: LLM: $O(|\text{instruction}| \cdot |\text{model}|)$ dominated by API call (100ms–5s). Keyword: $O(N_{\text{zones}} \cdot |\text{text}|)$. Levenshtein: $O(N_{\text{zones}} \cdot L^2)$.

- LLM has correct world knowledge about zone names and spatial relationships
- Confidence c calibrated to goal quality
- zone_map covers all deployment zones
- LLM API accessible (offline: keyword fallback only)

⑤ Research Meaning

Research Question

Can a structured-output LLM interface reliably parse natural language mission specs into ordered waypoint sequences, with confidence calibration sufficient to safely drive a navigation system?

Novelty

The keyword fallback ensuring offline-capable operation is a practical engineering contribution. Integration with confidence-gated VLM (C11) and topological map (C17) creates a complete language-to-navigation pipeline.

Related Work

SayCan (Ahn 2022). Code-as-Policies (Liang 2023). Inner Monologue (Huang 2022). ProgPrompt (Singh 2023).

⑥Results & Evidence

Offline keyword fallback validated. JSON parsing handles format variations. Levenshtein catches misspellings. □ Live LLM accuracy not measured in navigation.

Metrics Used

- Mission parse accuracy
- Zone identification accuracy
- API latency distribution
- Fallback rate

What Is Missing

No live LLM evaluation in navigation. No comparison with Code-as-Policies or SayCan. Confidence calibration not analyzed.

⑦Reviewer Critique

Weaknesses

- LLM confidence not reliably calibrated
- Keyword fallback: position heuristic fails for 'first B then A'
- LLM may hallucinate zone names not in map
- No temporal language: 'wait 5 minutes, then go to B'

Hidden Assumptions

- LLM correct knowledge of deployment environment
- Confidence $c \geq \tau_c$ implies correct parse
- zone_map complete

Failure Scenarios

- Ambiguous: 'room on the left' — which room?
- Sequential dependencies: 'pick up package at A, deliver to B'
- Unknown zones not in zone_map

⑧How to Turn This Into a Paper

Paper Title	Natural Language Mission Planning for Robot Navigation: Structured LLM Parsing with Confidence-Gated Fallback
Venue	ICRA NLP4Robots Workshop / HRI

Missing Experiments

- 100 natural language instructions → measure parse accuracy
- Comparison with Code-as-Policies on same instructions
- Confidence calibration analysis
- Temporal instruction extension

Ablation Studies Needed

- JSON-structured vs. free-text prompting
- τ_c sensitivity: false negative vs. bad parse acceptance
- Keyword fallback vs. LLM offline
- With vs. without fuzzy matching

9 Open Research Questions

72. Can LLM be fine-tuned on deployment-specific vocabulary?
73. How to handle ambiguous instructions — clarifying questions?
74. Can LLM generate executable planning code (Code-as-Policies)?
75. Can LLM use current map state to ground spatial references?

10 Two-Minute Explanation for Anyone

You should be able to tell a robot 'check the cafeteria, go to storage, return to base' — just like giving directions to a person. This interface sends your instruction to a large language model, which converts it into a structured list of ordered waypoints. If the model is unavailable or not confident, a simpler keyword-scanning backup extracts room names from the text. Either way, the robot gets an executable mission. Key safety feature: low-confidence outputs are rejected in favor of the simpler but more reliable fallback.

Evaluation Scores

Dimension	Score	Comment
Originality	6/10	Incremental
Technical Depth	6/10	Prototype level
Publishability	5/10	Concept only
PhD Potential	6/10	Supporting work

Contribution 20 Multimodal Failure Explainer (XAI Diagnostics)

Category	Explainable AI / Diagnostics
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

When a robot fails, diagnosing the cause requires hours of manual log analysis. This module automates failure diagnosis: given a failure event, it produces a structured Markdown report explaining what happened, why, and what to fix — fusing visual, causal, and temporal-logic evidence.

Why does this problem exist?

As robots become more complex (neural networks, RL, diffusion models), their failure modes become harder to diagnose. Automated diagnosis scales with fleet size and enables systematic failure databases.

Why is it hard?

Fusing multiple evidence sources (visual scene, causal attribution, STL violations) into a coherent narrative requires careful integration. Report quality depends entirely on the underlying modules (VLM, SCM, STL).

② Intuition-First Explanation

When a surgical robot has an unexpected outcome, specialists review video, sensor data, and procedure logs to produce an incident report. This module automates that review for navigation robots: watch the failure, trace the causal chain, identify violated specifications, recommend fixes.

Analogy: Aircraft flight data recorders are analyzed after accidents by automated systems before human investigators. This module is the automated black-box analyst for navigation robots, producing a first-pass diagnosis within seconds.

③ What I Built — Step by Step

- FailureEvent dataclass: type, timestamp, position, velocity, RGB frame, trajectory, STL trace, sensor readings
- FailureReport dataclass: scene description, causal attribution, STL violations, corrective actions, confidence
- VLM scene description (C11): RGB frame → text if available
- Causal attribution (C14): SCM root_cause_ranking() → top causes
- STL violation summary: extract $\{(\text{spec}, p) : p < 0\}$ from trace
- Corrective action dictionary: type-indexed recommendations
- Markdown and JSON export
- 4 tests passing

Data Flow

FailureEvent → [VLM scene | SCM causes | STL violations] → assemble *FailureReport* → Markdown/JSON

④ Deep Technical Breakdown

Key Components

- Scene describer: VLM if available, rule-based fallback
- Causal attributer: SCM from C14
- STL summarizer: negative p values from C18 monitor
- Action recommender: curated dictionary indexed by failure type
- Report exporter: Markdown and JSON formats

Pseudocode / Algorithm

```
EXPLAIN(event: FailureEvent) -> FailureReport:

// Step 1: Scene description
if event.frame and vlm_available:
    scene = vlm.describe(event.frame)
else:
    scene = rule_based(event.position, event.velocity, event.sensor_readings)

// Step 2: Causal attribution
causes = scm.root_cause_ranking(event.sensor_readings, n_samples=50)

// Step 3: STL violations
violations = [(s, r) for s, r in event.stl_trace.items() if r < 0]

// Step 4: Corrective actions
actions = ACTION_DICT[event.failure_type]

return FailureReport(scene, causes[:4], violations, actions)
```

Mathematical Formulation

```
Scene: VLM(If) → text or rule_based(sf, ṡf, zf) → text
Causes: φi from C14 root_cause_ranking
Violations: V = {(φi, ρi) : ρi(x, t) < 0} from C18 STL monitor
Report confidence: 0.4 + 0.1·1[causal_available] + 0.1·1[vlm_available]
```

Complexity & Assumptions

Time complexity: $O(N_{scm} \cdot |graph|)$ for causal + $O(|STL_specs|)$ for violations. Dominated by SCM Monte Carlo.

- VLM scene description informative
- SCM correctly models causal relationships (from C14)
- STL trace was recorded during episode (C18 active)
- Corrective action dictionary complete and correct

⑤ Research Meaning

Research Question

Can automated fusion of visual, causal, and temporal-logic evidence produce navigation failure reports that engineers rate as more diagnostic and actionable than raw log analysis?

Novelty

Combining VLM scene description, SCM causal attribution, and STL monitoring in a unified failure diagnosis pipeline is novel in robot navigation. Each component individually is known; their systematic integration for navigation diagnostics is the contribution.

Related Work

Richens et al. (2020) counterfactual diagnosis in medicine. Arrieta et al. (2020) XAI survey. Li et al. (2021) causal analysis for program repair.

⑥ Results & Evidence

Reports generated correctly for COLLISION, TIMEOUT, SENSOR_FAULT. Markdown output human-readable. □ No human evaluation study.

Metrics Used

- Human evaluation: usefulness rating vs. raw logs
- Causal attribution accuracy
- Report generation latency
- Quality vs. available evidence

What Is Missing

No human evaluation. Action dictionary manually curated. VLM depends on live API. No comparison with alternative diagnostic methods.

⑦ Reviewer Critique

Weaknesses

- Without human evaluation: unknown whether reports are useful
- Corrective action dictionary static — not learned
- Causal attribution quality inherited from C14 limitations
- Report confidence heuristic not calibrated

Hidden Assumptions

- VLM accurate scene description
- STL trace recorded during failure episode
- Action dictionary complete

Failure Scenarios

- Novel failure types not in dictionary
- Poor VLM description (bad lighting)
- Multiple simultaneous causes
- Episode without RGB frame

⑧ How to Turn This Into a Paper

Paper Title	Multimodal Navigation Failure Diagnosis: Fusing Visual, Causal, and Temporal-Logic Evidence into Actionable Reports
Venue	ICRA XAI Workshop / HRI / RA-L

Missing Experiments

- Human evaluation: engineer comparison of automated vs. manual log analysis
- Ground-truth causal study: inject known failures, evaluate attribution accuracy
- Learn corrective actions from successful repairs
- LLM-only vs. full multimodal pipeline comparison

Ablation Studies Needed

- With vs. without causal attribution: does it improve engineer actions?
- With vs. without VLM scene: does visual context help?
- Full report vs. STL violations only vs. causal only
- Rule-based vs. LLM fallback for scene description

9 Open Research Questions

76. Can corrective action dictionary be learned from past failures and repairs?
77. Can report be interactive: 'what if X had been different?'
78. Can reports be shared across fleet to build institutional knowledge?
79. Can predictive failure detector be built from report database?

10 Two-Minute Explanation for Anyone

When a robot fails, diagnosing why usually takes engineers hours of log analysis. This module automates the process: given a failure, it automatically (1) describes the visual scene using a language model, (2) identifies the most causally responsible navigation variable using causal reasoning, (3) lists which safety specifications were violated, and (4) recommends specific fixes. Output is a structured Markdown report that can be read immediately or filed for future reference. Think of it as an automated incident report system for robots.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 21 PPO Navigation Agent

Category	Reinforcement Learning / Navigation
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Classical planners require explicit maps. RL agents learn navigation policies from trial-and-error experience. This module implements PPO with risk-shaped rewards: penalties for proximity to obstacles encourage safe navigation without explicitly programming obstacle avoidance rules.

Why does this problem exist?

RL can discover navigation policies that classical methods miss — especially in complex dynamic environments. Risk-shaped rewards connect DynNav's probabilistic risk estimation to the RL training signal.

Why is it hard?

PPO requires backpropagation through actor and critic networks. Numpy stubs cannot implement backpropagation. The current implementation demonstrates the full PPO data flow but does not actually update the policy.

② Intuition-First Explanation

Instead of programming explicit rules ('if obstacle within 0.5m, turn away'), the robot learns by trying things and receiving rewards (goal reached) and penalties (got too close to obstacle). Over thousands of episodes, it learns which behaviors lead to success. PPO structures this learning to be stable and efficient.

Analogy: Teaching a dog to fetch by rewarding success and ignoring failure — not by explicitly explaining the physics of ball trajectories. The dog learns from experience what works. PPO is the mathematical formalization of this reward-based learning for robots.

③ What I Built — Step by Step

- Risk-shaped reward: $r = -\text{step_penalty} + \text{progress_bonus} - \text{obstacle_proximity_penalty} - \text{collision_penalty} + \text{goal_bonus}$
- Actor network: MLP $\rightarrow$ (action_mean, action_std) — Gaussian policy
- Critic network: MLP $\rightarrow V(s)$ — state value function
- Rollout buffer: collect T steps, store (s, a, log_pi, r, V, done)
- GAE: $\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}$, $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
- PPO clip: $L = E[\min(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t, 1 \pm \epsilon) \cdot \hat{A}_t)]$
- ☐ Actor/Critic are numpy MLP stubs — no backprop $\rightarrow$ policy NOT updated
- NavEnv: custom navigation environment with risk-shaped rewards

Data Flow

NavEnv → rollout collection (T steps) → GAE advantage computation → PPO clipped update (▢ stub) → new policy

④ Deep Technical Breakdown

Key Components

- NavEnv: grid environment, risk-shaped reward, 14-dim observation (pos, goal_dir, clearances)
- ActorNetwork: MLP → action distribution (tanh mean, log std)
- CriticNetwork: MLP → scalar value $V(s)$
- RolloutBuffer: stores T steps, computes GAE
- PPO updater: K epochs of clipped objective ▢ STUB

Pseudocode / Algorithm

```
ROLLOUT COLLECTION:
obs = env.reset()
for t in range(T):
    a, log_pi, V = actor_critic(obs)
    obs_next, r, done, _ = env.step(a)
    buffer.add(obs, a, log_pi, r, V, done)
    obs = obs_next if not done else env.reset()
advantages, returns = buffer.compute_gae(last_V)

PPO UPDATE (K epochs):
for minibatch in buffer:
    new_log_pi, new_V = actor_critic(minibatch.obs)
    ratio = exp(new_log_pi - minibatch.log_pi)
    L_clip = min(ratio*Adv, clip(ratio, 1-eps, 1+eps)*Adv)
    L_total = -L_clip + c_v*(new_V - Returns)^2 - c_e*entropy
    [BACKPROP L_total] // ▢ NOT IMPLEMENTED (numpy stubs)
```

Mathematical Formulation

```
GAE:  $\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}$ ,  $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 
PPO clip:  $L^{\text{CLIP}}(\theta) = E[\min(r_t \cdot \hat{A}_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)]$ 
Ratio:  $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ 
Total:  $L = -L^{\text{CLIP}} + c_v \cdot L^{\text{V}} - c_e \cdot H[\pi_\theta]$ 
Risk reward:  $r(s, a) = -0.01 + 0.1/(d_{\text{goal}} + \epsilon) - 0.5 \cdot \max(0, d_{\text{safe}} - d_{\text{obs}}) - 5 \cdot 1[\text{col}] + 10 \cdot 1[\text{goal}]$ 
```

Complexity & Assumptions

Time complexity: Rollout: $O(T \cdot |\text{env}|)$. GAE: $O(T)$. PPO update: $O(K \cdot T / \text{batch} \cdot |\text{networks}|)$. All fast with stubs.

- Actor/Critic trainable via backprop ▢ numpy stubs do not support this
- Risk-shaped reward sufficient to learn safe navigation
- NavEnv representative of real navigation scenarios
- PPO hyperparameters (ϵ , γ , λ , c_v , c_e) appropriately tuned

⑤ Research Meaning

Research Question

Does risk-shaped reward incorporating CVaR-inspired obstacle proximity penalties learn safer navigation policies than standard goal-reaching reward, across diverse unknown environments?

Novelty

Risk-shaped reward design connecting DynNav's CVaR risk metrics to RL training signal. The integration with CBF safety shield (C18) as a training wrapper and curriculum learning (C22) for staged training is the novel systems contribution.

Related Work

Schulman et al. (2017) PPO. Mirowski et al. (2017) navigation with RL. Savva et al. (2019) Habitat. Henderson et al. (2018) deep RL reproduced.

⑥ Results & Evidence

Rollout collection executes. GAE computed correctly. Clipped objective evaluated. ☐ Policy NOT updated — training loop is non-functional without backprop.

Metrics Used

- Mean episode reward
- Episode length
- Value loss over updates
- Collision rate during training

What Is Missing

Policy not actually trained. No comparison with classical A* baseline. No curriculum integration evaluation. No hardware deployment.

⑦ Reviewer Critique

Weaknesses

- ☐ Numpy stubs: no gradient computation → policy never updates → this is NOT a trained RL agent
- NavEnv not validated against real TurtleBot3 dynamics
- No comparison with standard reward (goal only) to show risk-shaping helps
- Random policy performance not characterized as lower bound

Hidden Assumptions

- Backpropagation implementable with numpy (false)
- NavEnv representative of real navigation
- Risk reward parameters correctly tuned

Failure Scenarios

- ☐ The entire training loop is a framework demonstration only — policy never improves
- Risk penalty too high: robot refuses to move
- GAE γ and λ not tuned for navigation episodes

⑧ How to Turn This Into a Paper

Paper Title	Risk-Shaped Proximal Policy Optimization for Safe Autonomous Navigation in Unknown Environments
Venue	ICRA / CoRL / RA-L — requires PyTorch implementation first

Missing Experiments

- Rewrite in PyTorch: proper actor-critic with backpropagation
- Train for 1M+ timesteps on NavEnv
- Compare: risk-shaped reward vs. standard goal-only reward on same environment
- Integrate CBF shield (C18) during training: does it improve final policy safety?

Ablation Studies Needed

- Risk reward vs. goal-only reward vs. shaped + safety penalty
- GAE λ sensitivity on navigation episodes
- PPO clip ϵ : conservative vs. aggressive policy updates
- With vs. without curriculum (C22) for staged training

9 Open Research Questions

80. Can risk-shaped reward be derived from the CVaR A* cost function automatically?
81. Can CBF (C18) be used as a hard constraint during RL training (safe RL)?
82. Can meta-RL (MAML) learn a policy that adapts to new environments in K steps?
83. How does risk shaping affect sample efficiency compared to reward shaping?

10 Two-Minute Explanation for Anyone

Instead of programming explicit navigation rules, the robot learns by trial and error. It tries different actions, gets rewards for reaching the goal and penalties for getting close to obstacles. The PPO algorithm structures this learning to be stable. The key innovation is the risk-shaped reward: it penalizes obstacle proximity proportionally, connecting DynNav's probabilistic risk reasoning to the training signal. Critical caveat: the neural network stubs do not support backpropagation, so the policy never actually learns — this is a training framework awaiting PyTorch implementation.

Evaluation Scores

Dimension	Score	Comment
Originality	6/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	5/10	Concept only
PhD Potential	7/10	Supporting work

Contribution 22 Curriculum RL Training

Category	Reinforcement Learning / Training Methodology
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Training RL agents in complex environments from the start is sample-inefficient: the agent rarely reaches the goal, gets mostly negative rewards, and learns slowly. Curriculum learning gradually increases difficulty as the agent improves — matching training difficulty to current capability.

Why does this problem exist?

Without curriculum, a navigation agent trained in complex environments (many obstacles, long distances, sensor noise) spends most episodes failing with no gradient signal. Curriculum ensures the agent always practices at a productive difficulty level.

Why is it hard?

Defining the right difficulty progression and advancement criteria requires domain knowledge. The curriculum logic is separate from the RL algorithm — it must correctly identify when the agent has mastered each level. Connecting to the PPO stub means the RL backend is non-functional.

② Intuition-First Explanation

You do not teach swimming by throwing a child into the deep end. You start in the shallow end, progress to deeper water as confidence grows. The curriculum does the same for the robot: easy maps first (few obstacles, close goal), then harder as success rate improves.

Analogy: A video game difficulty system that adapts to the player's skill level. New players see easier enemies; experts see full difficulty. The curriculum scheduler is that adaptive difficulty system — except for robot navigation training.

③ What I Built — Step by Step

- 5-stage difficulty ladder: easy (2 obs, 5×5m, static) → medium → hard → expert → extreme (25 obs, 20×20m, 0.5m/s noise)
- Difficulty dimensions: N_obstacles, map_size, obstacle_speed, sensor_noise, goal_distance_range
- Rolling success rate: $SR_W = \text{mean}(\text{success}[-W:])$
- Adaptive advancement: advance if $SR_W \geq \tau_stage$ AND $stage < 4$
- 3 strategies: ADAPTIVE, FIXED (every N episodes), REVERSE (start near goal)
- CurriculumNavEnv: wraps NavEnv, injects difficulty parameters transparently
- Stage history logged with transition episodes and success rates
- 5 tests passing including stage advancement validation

Data Flow

Current difficulty stage $\rightarrow$ CurriculumNavEnv parameters $\rightarrow$ episode $\rightarrow$ success/fail $\rightarrow$ SR_W update $\rightarrow$ advance if $SR_W \geq \tau \rightarrow$ next stage

④ Deep Technical Breakdown

Key Components

- Difficulty ladder: 5 DifficultyLevel objects with parameter specs
- CurriculumScheduler: tracks SR_W, decides advancement
- CurriculumNavEnv: applies difficulty to NavEnv via parameter injection
- Stage logger: records transitions with episode numbers and SR at transition

Pseudocode / Algorithm

```
CURRICULUM SCHEDULER:
    outcomes = deque(maxlen=W)
    stage = 0

    def record_episode(success):
        outcomes.append(success)
        SR = mean(outcomes)
        if SR >= tau_stage[stage] and stage < 4:
            stage += 1
            outcomes.clear() // reset for new stage
            log_transition(stage-1, stage, SR)

CURRICULUM NAV ENV:
    def reset():
        d = scheduler.current_difficulty()
        set N_obstacles = d.n_obs
        set map_size = d.map_size
        set obs_speed = d.obstacle_speed
        set noise_std = d.sensor_noise
        set goal_range = d.goal_dist_range
        return initialize_episode()
```

Mathematical Formulation

```
SR_W(t) = (1/W)  $\sum_{i=t-W+1}^t 1[\text{success}_i]$ 
Advance:  $SR\_W(t) \geq \tau\_d$  AND  $stage\_t < 4$ 
Hysteresis: outcomes.clear() on advance  $\rightarrow$  prevents immediate re-advance
Reverse curriculum: start_range(t) expands as SR_W improves
```

Complexity & Assumptions

Time complexity: $O(1)$ per episode for SR update and advancement check. Negligible overhead.

- 5 stages cover the full difficulty range
- τ_stage thresholds appropriate for each stage
- W window size sufficient for stable SR estimate
- RL backend functional $\square$ PPO stub is not

⑤ Research Meaning

Research Question

Does adaptive curriculum scheduling reduce the number of environment interactions required to achieve a target navigation success rate in complex unknown environments?

Novelty

The 5-stage difficulty ladder is well-specified across 5 dimensions simultaneously. The adaptive advancement with hysteresis is a practical improvement over fixed-schedule curriculum. Connection to PPO (C21) and CBF (C18) for safe curriculum training is a novel systems combination.

Related Work

Bengio et al. (2009) curriculum learning. Narvekar et al. (2020) curriculum RL survey. Florensa et al. (2017) reverse curriculum. Portelas et al. (2020) teacher-student curriculum.

⑥ Results & Evidence

Curriculum logic executes correctly. Stage advancement from easy → medium validated. 5 tests passing. □ Full training requires functional PPO (C21).

Metrics Used

- Episodes to target success rate with vs. without curriculum
- Stage advancement episodes
- Success rate at each stage
- Final policy collision rate

What Is Missing

RL backend (C21) non-functional. Curriculum effectiveness not measurable without actual training. No comparison with flat training.

⑦ Reviewer Critique

Weaknesses

- Curriculum logic correct but connected to non-functional PPO stub
- τ_{stage} thresholds manually set — no principled method
- 5 stages may not cover all relevant difficulty dimensions
- No comparison with flat training to show curriculum benefit

Hidden Assumptions

- τ_{stage} thresholds correctly set
- 5 stages cover full difficulty range
- W sufficient for stable SR estimate

Failure Scenarios

- τ_{stage} too high: agent stuck at one stage forever
- τ_{stage} too low: agent advances before mastering stage
- Curriculum does not transfer to real robot (sim-to-real gap)

⑧ How to Turn This Into a Paper

Paper Title	Multi-Dimensional Adaptive Curriculum Learning for Safe Navigation RL in Dynamic Unknown Environments
Venue	ICRA / RA-L — requires functional PPO

Missing Experiments

- Train with functional PyTorch PPO: curriculum vs. flat training
- Measure: episodes to 70% success rate with and without curriculum
- Compare: ADAPTIVE vs. FIXED vs. REVERSE strategies
- Hardware: does curriculum-trained policy transfer to real TurtleBot3?

Ablation Studies Needed

- Adaptive vs. fixed schedule vs. reverse curriculum
- 5 stages vs. 3 stages vs. continuous difficulty
- W sensitivity: rolling window size effect
- τ _stage sensitivity: advancement threshold effect

⑨ Open Research Questions

84. Can difficulty be increased along only the dimension the agent struggles with (per-dimension curriculum)?
85. Can curriculum be coordinated across a fleet (C16 federated curriculum)?
86. Can the teacher (curriculum selector) be learned by another RL agent (teacher-student RL)?
87. What is the theoretical sample complexity reduction from curriculum in navigation?

⑩ Two-Minute Explanation for Anyone

Training a robot to navigate directly in complex environments is like throwing a beginner swimmer into the deep end — they fail constantly and learn nothing. My curriculum starts with simple environments (two obstacles, close goal, no noise) and only advances to harder settings when the robot succeeds reliably. Five difficulty stages cover all relevant dimensions: obstacle count, map size, obstacle speed, sensor noise, and goal distance. The advancement is automatic, based on rolling success rate. The curriculum logic is correct and tested; it awaits connection to a fully functional RL training backend.

Evaluation Scores

Dimension	Score	Comment
Originality	6/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	6/10	Needs experiments

PhD Potential	7/10	Supporting work
---------------	------	-----------------

Contribution 23 Gaussian Splatting Mapper (3D-GS)

Category	3D Mapping / Incremental Mapping
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Dense metric grids are memory-intensive and do not encode uncertainty naturally. 3D Gaussian Splatting provides a compact, differentiable scene representation where each Gaussian encodes position, shape, opacity, color, and confidence. This module builds such a map incrementally from streaming point clouds.

Why does this problem exist?

3DGS is the current state-of-the-art for scene representation (Kerbl 2023). A navigation-centric adaptation that extracts occupancy, uncertainty, and frontiers from Gaussians provides richer information than voxel grids while remaining compact.

Why is it hard?

Full 3DGS requires neural rendering (rasterization + RGB loss backpropagation). This module implements only the navigation-relevant components (occupancy extraction, uncertainty, frontiers), not photorealistic rendering.

② Intuition-First Explanation

Instead of storing every centimeter as occupied/free, represent the environment as a collection of 3D blobs — each describing a local region of space: where it is, what shape it has, how solid it appears, how confident we are. New observations either update existing blobs or create new ones.

Analogy: An artist sketching a landscape does not draw every leaf and stone — they use broad strokes to capture key features, with more detail where it matters. Gaussian Splatting is that artistic sketch of space: compact, expressive, and easy to update as new observations arrive.

③ What I Built — Step by Step

- Gaussian primitive: $(\mu_i, \Sigma_i, \alpha_i, c_i)$ — position, covariance, opacity, color
- Add frame: for each point, find nearest Gaussian by Mahalanobis distance
- Merge ($d_M < \tau$): EKF-style update of μ , Σ , confidence
- Add new: if no nearby Gaussian and $|G| < N_{\max}$, create new
- Confidence decay: $\text{conf} *= p$ each frame, prune below threshold
- 2D projection: project Gaussians to floor plane $\rightarrow$ occupancy grid $O(r,c)$
- Uncertainty map: $1 - \max_{\text{conf}}$ per cell
- Frontier detection: cells adjacent to high-uncertainty unoccupied cells
- 5 tests passing

Data Flow

Point cloud frame → Mahalanobis search → merge or add Gaussian → confidence decay + prune → project to 2D → O, U, frontiers

④ Deep Technical Breakdown

Key Components

- Gaussian store: list of $(\mu, \Sigma, \alpha, c, \text{conf})$ primitives
- Mahalanobis matcher: $d_M = \sqrt{(p-\mu)^T \Sigma^{-1} (p-\mu)} < \tau_{\text{merge}}$
- EKF updater: $\mu \leftarrow (1-\alpha)\mu + \alpha p, \Sigma \leftarrow (1-\alpha)\Sigma + \alpha(p-\mu)(p-\mu)^T$
- Confidence decayer: $\text{conf} *= \rho$ each frame
- 2D projector: Gaussian alpha-blending onto floor grid
- Frontier detector: boundary of known/unknown space

Pseudocode / Algorithm

```
ADD FRAME(points, pose):
    if pose: transform points to world frame
    decay_all_confidences(rho)
    for p in points:
        G_near = find_nearest_mahalanobis(p, gaussians, tau_merge)
        if G_near:
            G_near.mu = (1-alpha)*G_near.mu + alpha*p
            G_near.Sigma = (1-alpha)*G_near.Sigma + alpha*outer(p-G_near.mu)
            G_near.conf = min(1.0, G_near.conf + 0.05)
        elif len(gaussians) < N_max:
            gaussians.append(new_gaussian(p))
    prune(conf < conf_min)

2D OCCUPANCY:
    O = zeros(H, W)
    for G in gaussians:
        if z_min <= G.mu[2] <= z_max:
            cx, cy = project_to_grid(G.mu[:2])
            for (dx,dy) in neighborhood(sigma=sqrt(trace(G.Sigma))):
                O[cy+dy, cx+dx] = min(1, O[cy+dy, cx+dx] + G.alpha * gaussian_weight(dx,dy))
```

Mathematical Formulation

```
Gaussian:  $G_i = (\mu_i \in \mathbb{R}^3, \Sigma_i \in \mathbb{R}^{\{3 \times 3\}}, \alpha_i \in [0,1], c_i \in \mathbb{R}^3, \text{conf}_i \in [0,1])$ 
Mahalanobis:  $d_M(p,G) = \sqrt{(p-\mu)^T \Sigma^{-1} (p-\mu)}$ 
EKF merge:  $\mu \leftarrow (1-\alpha)\mu + \alpha p, \Sigma \leftarrow (1-\alpha)\Sigma + \alpha(p-\mu)(p-\mu)^T$ 
Conf decay:  $\text{conf}_i^{(t)} = \rho \cdot \text{conf}_i^{(t-1)}, \rho \in (0,1)$ 
2D projection:  $O(r,c) = \min(1, \sum_{G: z_i \in [z_{\min}, z_{\max}]} \alpha_i \cdot \exp(-d_{\{2D\}}^2 / 2\sigma_i^2))$ 
Uncertainty:  $U(r,c) = 1 - \max_{G \text{ near } (r,c)} \text{conf}_G$ 
```

Complexity & Assumptions

Time complexity: Add frame: $O(N_{\text{points}} \cdot |G|)$ naive, $O(N_{\text{points}} \cdot \log|G|)$ with spatial indexing.
Projection: $O(|G| \cdot \sigma_{\text{max}}^2)$. Frontier: $O(H \cdot W)$.

- Point clouds aligned to world frame (requires accurate pose)
- Mahalanobis distance sufficient for merge criterion
- 2D floor projection adequate (not 3D)

- Navigation-relevant components only — no neural rendering

⑤ Research Meaning

Research Question

Does an incremental 3DGS map provide richer uncertainty estimates and more useful frontier detection for navigation planning than standard 2D occupancy maps?

Novelty

Navigation-centric 3DGS — occupancy, uncertainty, and frontiers from Gaussian primitives — is a useful simplification of full 3DGS for robotics. EKF-style Gaussian fusion + confidence decay is a principled online update.

Related Work

Kerbl et al. (2023) 3DGS. Matsuki et al. (2024) MonoGS SLAM. Yugay et al. (2023) Gaussian-SLAM. OctoMap, Voxblox for comparison.

⑥ Results & Evidence

Occupancy extraction and frontier detection validated. Confidence decay and pruning verified. 5 tests passing.

Metrics Used

- Occupancy map accuracy vs. ground truth
- Frontier detection quality
- Memory efficiency vs. OctoMap at same accuracy
- Update throughput (frames/sec)

What Is Missing

No neural rendering. No comparison with OctoMap or Voxblox. No evaluation of occupancy accuracy vs. ground truth.

⑦ Reviewer Critique

Weaknesses

- Without neural rendering: not true 3DGS (no photorealistic view synthesis)
- No comparison with standard 2D occupancy maps
- Mahalanobis search naive: $O(|G|)$ per point — needs spatial indexing
- 2D floor projection ignores 3D vertical structure

Hidden Assumptions

- Pose estimate accurate for world frame transformation
- Mahalanobis sufficient for merge criterion
- Floor projection adequate for navigation

Failure Scenarios

- Poor pose estimation: Gaussians placed incorrectly
- Very cluttered environments: many small Gaussians, slow search

- Multi-story buildings: 2D floor projection inadequate

⑧ How to Turn This Into a Paper

Paper Title	Navigation-Centric 3D Gaussian Splatting: Online Occupancy and Uncertainty Extraction for Autonomous Exploration
Venue	ICRA / RA-L / 3DV

Missing Experiments

- Comparison: 3DGS occupancy vs. OctoMap vs. standard 2D grid on same datasets
- Add spatial indexing (k-d tree) and measure throughput
- Evaluate frontier quality: does 3DGS find better exploration targets than 2D grids?
- Full 3DGS: add neural rendering for RGB loss optimization

Ablation Studies Needed

- EKF merge vs. simple replacement vs. full Gaussian mixture update
- Confidence decay rate ρ sensitivity
- τ_{merge} threshold: merge vs. add tradeoff
- 2D projection resolution vs. accuracy

⑨ Open Research Questions

88. Can neural rendering be added efficiently for online SLAM?
89. Can Gaussians encode semantic labels (CLIP embeddings per Gaussian)?
90. How to handle multi-story environments with 3D planning?
91. Can Gaussian uncertainty directly feed CBF safety constraints?

⑩ Two-Minute Explanation for Anyone

Instead of storing the environment as a regular grid, this module represents it as a collection of 3D blobs — Gaussians — each describing a small region of space: where it is, what shape it has, how dense it appears, and how confident we are. New sensor readings either update existing blobs or create new ones. Low-confidence blobs fade over time and are pruned. The collection is then projected onto a 2D floor map for the path planner. Areas with no confident blobs are flagged as unexplored frontiers for targeted exploration.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	8/10	Full maths + code
Publishability	7/10	Needs experiments

PhD Potential	8/10	Strong chapter candidate
---------------	------	--------------------------

Contribution 24 NeRF Uncertainty Maps (MC-Dropout)

Category	3D Mapping / Uncertainty Quantification
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

NeRF models learn to render views of a scene but do not natively quantify which regions are well-observed vs. uncertain. MC-Dropout applied at inference produces epistemic uncertainty estimates, guiding exploration toward unobserved areas.

Why does this problem exist?

Standard NeRF gives a single rendering without confidence. For exploration planning, knowing WHERE the model is uncertain is as important as knowing WHERE obstacles are. High-uncertainty regions are the highest-value exploration targets.

Why is it hard?

NeRF inference is computationally expensive. MC-Dropout with T passes multiplies this cost by T . The NeRF must be trained on actual camera data — untrained weights produce random uncertainty values.

② Intuition-First Explanation

When you look at a half-finished jigsaw puzzle, you can immediately tell which parts are done (clear picture) and which are missing (gaps). A trained NeRF's rendering quality plays the same role: confident in seen areas, uncertain in unseen ones. MC-Dropout quantifies this uncertainty.

Analogy: A weather model is most confident for the next 24 hours (well-constrained by recent observations) and increasingly uncertain for day 7+ (little data). NeRF with MC-Dropout is similarly confident near well-observed viewpoints and uncertain far away.

③ What I Built — Step by Step

- Positional encoding: Fourier features $L=6$ frequencies for position, $L=4$ for direction
- TinyNeRF: MLP density/color network with random dropout masking
- MC-Dropout: $T=8$ forward passes per ray with different dropout masks
- Volume rendering: correct transmittance computation $\sum T_k \cdot (1 - \exp(-\sigma_k \cdot \delta_k)) \cdot c_k$
- Per-ray uncertainty: $\sum T_k \cdot (1 - \exp(-\sigma_k \cdot \delta_k)) \cdot \text{std_MC}[\sigma_k]$
- 2D projection: uncertainty aggregated onto navigation grid
- Exploration weights: $U(r,c) \cdot (1 - O(r,c))$ normalized
- ☐ NeRF weights random — uncertainty values meaningless without training

Data Flow

Camera poses → ray casting → TinyNeRF MC-Dropout (T passes) → density variance per point → volume rendering → uncertainty per ray → 2D grid → exploration weights

④ Deep Technical Breakdown

Key Components

- Positional encoder: Fourier features for (x,y,z) and direction
- TinyNeRF: 2-layer MLP with random dropout masking
- MC sampler: T random dropout masks per ray evaluation
- Volume renderer: transmittance × density × color summation
- Uncertainty aggregator: std over T density estimates
- 2D projector: hit-point uncertainty onto grid
- Weight normalizer: $U \cdot (1-O) / \text{sum}$

Pseudocode / Algorithm

```
MC-DROPOUT UNCERTAINTY PER RAY:
    t_vals = linspace(near, far, N_coarse)
    points = origin + t_vals * direction

    sigma_samples = []
    for _ in range(T): // T dropout passes
        sigma, _ = nerf_with_dropout(points, direction)
        sigma_samples.append(sigma)

    sigma_mean = mean(sigma_samples, axis=0)
    sigma_std = std(sigma_samples, axis=0)

    T_k = exp(-cumsum(sigma_mean * deltas)) // transmittance
    w_k = T_k * (1 - exp(-sigma_mean * deltas))
    uncertainty_ray = sum(w_k * sigma_std)

2D UNCERTAINTY MAP:
    for pose in camera_poses:
        for (u, v) in ray_grid:
            depth, _, unc = render_ray(origin, direction(u,v,pose))
            hit = project_3D(origin + depth*direction, grid_resolution)
            U_grid[hit.y, hit.x] += unc
```

Mathematical Formulation

```
NeRF rendering:  $C(r) = \sum_k T_k (1 - \exp(-\sigma_k \delta_k)) c_k$ 
Transmittance:  $T_k = \exp(-\sum_{j < k} \sigma_j \delta_j)$ 
MC variance:  $\text{Var}[\sigma(x)] \approx (1/T) \sum_t \sigma_t^2 - ((1/T) \sum_t \sigma_t)^2$ 
Ray uncertainty:  $u(r) = \sum_k w_k \cdot \text{std}_{MC}[\sigma_k]$ 
Exploration weights:  $w(r, c) \propto U(r, c) \cdot (1 - O(r, c))$ 
```

Complexity & Assumptions

Time complexity: NeRF: $O(T \cdot N_{\text{rays}} \cdot N_{\text{samples}} \cdot |\text{MLP}|)$ — T multiplies inference cost. Dominant cost for real deployment.

- MC-Dropout approximates Bayesian inference □ controversial assumption
- NeRF trained on relevant data □ random weights currently

- T=8 samples sufficient for variance estimate
- Dropout rate correctly set for meaningful uncertainty

⑤ Research Meaning

Research Question

Can MC-Dropout NeRF rendering variance serve as a reliable spatial uncertainty signal for exploration planning, superior to entropy-based occupancy uncertainty?

Novelty

Applying MC-Dropout uncertainty to NeRF for navigation exploration guidance is an active research direction. The navigation-centric 2D projection and integration with exploration weight computation is the contribution.

Related Work

Mildenhall et al. (2020) NeRF. Gal & Ghahramani (2016) MC-Dropout. Ran et al. (2023) ActiveRMAP. Shen et al. (2023) ORBIT.

⑥ Results & Evidence

Volume rendering mathematically correct. MC-Dropout produces variance estimates. 2D projection executes. □ Random NeRF weights → random uncertainty values.

Metrics Used

- Uncertainty map correlation with actual unobserved areas
- Exploration target quality: does high-U go to unexplored regions?
- Rendering PSNR (requires training)
- Inference latency per uncertainty map

What Is Missing

NeRF not trained. Uncertainty values meaningless. No comparison with entropy-based exploration (C07). No latency benchmarks.

⑦ Reviewer Critique

Weaknesses

- □ Untrained NeRF → random uncertainty maps → meaningless exploration guidance
- MC-Dropout as Bayesian approximation is controversial — may not give calibrated uncertainty
- NeRF inference latency likely too high for real-time navigation
- Requires dense camera coverage for accurate NeRF training

Hidden Assumptions

- MC-Dropout approximates true Bayesian NeRF uncertainty (controversial)
- Training data available for deployment environment
- T=8 samples sufficient for calibrated uncertainty

Failure Scenarios

- □ Without training: all outputs are random noise

- Sparse camera coverage: NeRF training fails
- Computational budget: $T \cdot N_{\text{rays}} \cdot N_{\text{samples}}$ may exceed real-time budget

⑧How to Turn This Into a Paper

Paper Title	NeRF-Driven Uncertainty Estimation for Active Robot Exploration
Venue	ICRA / 3DV / NeurIPS workshop — requires trained NeRF

Missing Experiments

- Train NeRF on TurtleBot3 RGB-D dataset
- Evaluate: does high-NeRF-uncertainty correlate with unexplored areas?
- Compare: NeRF uncertainty vs. occupancy entropy (C07) for exploration guidance
- Latency optimization: hash-encoding NeRF (instant-NGP) for real-time use

Ablation Studies Needed

- MC-Dropout $T=4$ vs. 8 vs. 16: uncertainty quality vs. compute
- NeRF uncertainty vs. occupancy entropy: which guides exploration better?
- With vs. without positional encoding
- Uncertainty weighting: U alone vs. $U \cdot (1-O)$

⑨Open Research Questions

92. Can uncertainty be obtained from NeRF without MC-Dropout (e.g., ensemble)?
93. How should NeRF uncertainty and occupancy entropy be combined?
94. Can instant-NGP make NeRF inference fast enough for real-time navigation?
95. Can semantic NeRF uncertainty guide open-vocabulary exploration (C17)?

⑩Two-Minute Explanation for Anyone

NeRF is a neural network that learns to render photos of a scene from any angle. In places the robot has seen many times, the network is very confident about what's there. In unexplored areas, it's uncertain. By running the network multiple times with random dropout (MC-Dropout), the variance between runs quantifies this uncertainty. This uncertainty map guides exploration: the robot goes to high-uncertainty areas. The framework is correctly implemented; the NeRF needs to be trained on the deployment environment's camera data before it produces meaningful uncertainty estimates.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	7/10	Prototype level

Publishability	6/10	<i>Needs experiments</i>
PhD Potential	7/10	<i>Supporting work</i>

Contribution 25 Adversarial Attack Simulator

Category	Cybersecurity / Robustness Evaluation
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

Navigation systems relying on neural networks for perception are vulnerable to adversarial attacks — small, crafted perturbations that cause incorrect decisions. This module generates both gradient-based attacks (FGSM, PGD) on neural inputs and physics-plausible sensor attacks (LiDAR spoofing, odometry drift) for systematic robustness evaluation.

Why does this problem exist?

As navigation systems incorporate more neural networks (VLM goals C11, NeRF C24, PPO policy C21), their vulnerability to adversarial inputs grows. Systematic attack simulation is prerequisite to hardening.

Why is it hard?

Gradient-based attacks require backpropagation. Finite-difference approximation (used here) scales as $O(d)$ — slow for high-dimensional inputs. Physics-plausible attacks must be realistic enough to be informative.

② Intuition-First Explanation

Security researchers must think like attackers to build defenses. This module implements the attacker's toolkit: crafted perturbations on neural inputs, phantom obstacles injected into LiDAR, and gradual position corruption. By testing these attacks, we learn where the navigation system is brittle.

Analogy: Penetration testers at a bank try to break in before real attackers can. This module is the penetration tester for the navigation system — systematically attempting every known attack type to identify vulnerabilities before deployment.

③ What I Built — Step by Step

- FGSM: $x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L)$ via finite differences
- PGD: iterative FGSM with L^∞ projection, random start
- LiDAR phantom injection: N clustered points near robot position
- LiDAR point removal: random or targeted fraction removal
- LiDAR sector blinding: zero out angular range
- Odometry drift: Gaussian random walk on position estimate
- RobustnessEvaluator: runs full attack suite, reports metrics
- 6 tests passing including ϵ -bound verification

Data Flow

Observations → attack (FGSM/PGD/LiDAR/odom) → corrupted observations → navigation system → compare degradation

④ Deep Technical Breakdown

Key Components

- GradientAttacker: FGSM and PGD via finite differences
- LiDARAttacker: phantom injection, point removal, sector blinding
- OdometrySpoof: Gaussian drift with activation/deactivation
- RobustnessEvaluator: runs all attacks, aggregates metrics

Pseudocode / Algorithm

```
FGSM:
grad = finite_diff_gradient(x, loss_fn, h=1e-4)  // O(d) queries
x_adv = clip(x + eps*sign(grad), 0, 1)
assert max(abs(x_adv - x)) <= eps

PGD:
x_adv = x + uniform(-eps, eps)  // random start
for k in range(pgd_steps):
    grad = finite_diff_gradient(x_adv, loss_fn)
    x_adv = project_L_inf(x_adv + alpha*sign(grad), x, eps)

LIDAR PHANTOM INJECTION:
for _ in range(N_phantoms):
    angle = uniform(0, 2*pi)
    dist = uniform(0.5, max_range*0.6)
    phantom_cluster = robot_pos + dist*[cos, sin, 0] + normal(0, 0.05, (8,3))
    point_cloud = vstack([point_cloud, phantom_cluster])

ODOM DRIFT:
delta += normal(0, sigma_d)  // accumulate random walk
odom_corrupted = odom_true + delta
```

Mathematical Formulation

```
FGSM:  $x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x L(f_\theta(x), y))$ ,  $\|x_{adv} - x\|_\infty \leq \epsilon$ 
PGD:  $x^{k+1} = \Pi_{B_\epsilon(x)}(x^k + \alpha \cdot \text{sign}(\nabla_{x^k} L))$ 
Finite diff:  $\partial L / \partial x_i \approx (L(x+h \cdot e_i) - L(x-h \cdot e_i)) / (2h)$  — O(d) queries
Odom drift:  $\delta_t = \delta_{t-1} + \eta_t$ ,  $\eta_t \sim N(0, \sigma_d^2 I)$  (Gaussian random walk)
Phantom:  $p_i = p_{robot} + r_i \cdot [\cos \theta_i, \sin \theta_i, 0]^T + \epsilon_i$ ,  $\epsilon_i \sim N(0, 0.05^2 I)$ 
```

Complexity & Assumptions

Time complexity: FGSM: $O(d \cdot 2)$ evaluations (finite diff). PGD: $O(K \cdot d \cdot 2)$. LiDAR: $O(N_{phantoms})$. Odom: $O(1)$. Gradient attacks scale poorly with d .

- Finite differences approximate true gradient (valid for smooth loss)
- ϵ -bounded attacks remain in valid input range
- LiDAR phantom clusters are physics-plausible
- Odometry drift follows Gaussian random walk

⑤ Research Meaning

Research Question

What is the adversarial robustness of DynNav's neural navigation components to gradient-based and physics-plausible sensor attacks, and does the IDS (C08) detect these attacks?

Novelty

Comprehensive attack suite — gradient attacks + physics-plausible sensor attacks — evaluated against the same navigation system, connected to the IDS (C08) for detection evaluation. Most robustness work evaluates one attack type.

Related Work

Goodfellow et al. (2015) FGSM. Madry et al. (2018) PGD. Cao et al. (2019) LiDAR attacks. Eykholt et al. (2018) physical adversarial examples.

⑥ Results & Evidence

FGSM and PGD produce measurably higher loss values. LiDAR phantoms add correct number of points. Odometry drift accumulates correctly. ϵ -bounds verified in 6 tests.

Metrics Used

- FGSM/PGD: loss increase per attack
- LiDAR: points added/removed fraction
- Odom: drift magnitude after N steps
- IDS detection rate under each attack

What Is Missing

Finite differences not scalable to real neural networks. No hardware attack experiments. No connection to IDS detection evaluation (C08) in current experiments.

⑦ Reviewer Critique

Weaknesses

- Finite differences $O(d)$ — impractical for high-dimensional neural networks
- FGSM/PGD without backprop = very approximate — cannot fully test neural vulnerability
- No end-to-end evaluation: how does attack affect final navigation decisions?
- IDS (C08) evaluation against these attacks not yet done

Hidden Assumptions

- Finite differences approximate true gradient (may fail for non-smooth loss)
- ϵ -bounded perturbations remain meaningful for navigation
- Phantom clusters are physically plausible

Failure Scenarios

- High-dimensional observations: finite differences $O(d)$ too slow
- Adaptive attackers knowing the defense scheme
- Replay attacks: historically valid readings

⑧ How to Turn This Into a Paper

Paper Title	Adversarial Robustness Evaluation for Autonomous Navigation: From Gradient Attacks to Physics-Plausible Sensor Manipulation
Venue	ICRA Security Workshop / CPS-IoTSec

Missing Experiments

- Connect to IDS (C08): measure detection rate for each attack type
- Replace finite differences with PyTorch autograd for real neural networks
- Physical attack: real LiDAR interference on TurtleBot3
- End-to-end: does attack X cause collision rate to increase? By how much?

Ablation Studies Needed

- FGSM vs. PGD: which attacks are harder to defend?
- ϵ sensitivity: how small must ϵ be for attacks to be imperceptible?
- LiDAR phantom vs. removal vs. blinding: which degrades navigation most?
- With vs. without IDS (C08): does detection prevent navigation failure?

9 Open Research Questions

96. Can adversarial training using these attacks improve navigation robustness?
97. Can physical adversarial patches on obstacles fool the VLM agent (C11)?
98. How do gradient attacks interact with the CBF safety shield (C18)?
99. Can certified robustness bounds be computed for the navigation pipeline?

10 Two-Minute Explanation for Anyone

Security researchers must think like attackers. This module implements the attacker's toolkit for robot navigation: it crafts small perturbations to confuse neural network components (FGSM/PGD), injects phantom obstacles into LiDAR scans, removes real ones, and gradually corrupts the robot's sense of position. By systematically applying these attacks, we identify where the navigation system is most brittle — and can then test whether the intrusion detection system (C08) catches them. The physics-plausible sensor attacks are especially realistic and valuable for real-world security evaluation.

Evaluation Scores

Dimension	Score	Comment
Originality	7/10	Incremental
Technical Depth	7/10	Prototype level
Publishability	7/10	Needs experiments
PhD Potential	7/10	Supporting work

Contribution 26 Byzantine Fault-Tolerant Swarm Consensus

Category	Multi-Robot / Distributed Systems
Background colors	indicate topic cluster

① Explain Like I'm a Beginner

What problem does this solve?

In a robot fleet, some robots may be hacked, broken, or reporting wrong information. A naive majority vote would be corrupted. Byzantine Fault Tolerance (BFT) enables the honest majority to reach a correct collective decision even when up to 1/3 of robots are lying.

Why does this problem exist?

As robot fleets grow and operate in adversarial environments, compromised robots become a realistic threat. Naive coordination (mean of all proposals) is trivially corrupted by a single Byzantine robot. BFT provides mathematical guarantees.

Why is it hard?

Full Byzantine fault tolerance (PBFT) requires multiple communication rounds and cryptographic message authentication. This module implements a statistical approximation (MAD-based outlier detection + weighted median) that achieves practical robustness without cryptographic overhead.

② Intuition-First Explanation

In a group of 6 friends voting on a restaurant, one friend has been bribed to suggest the worst option. If you average all suggestions, the bribe works. If you use the median (most common choice), the briber needs to control at least half the friends — much harder. Weighted median is even more robust.

Analogy: Stock market consensus prices are not simple averages — outlier trades are excluded from index calculations. Financial markets have developed robust aggregation mechanisms over centuries to resist manipulation. This module applies the same principle to robot navigation planning.

③ What I Built — Step by Step

- SwarmRobot: runs local A^* → proposal $P_k = (\text{path}, \text{cost}, \text{confidence})$
- Fault modes: random (random path+cost), constant_bad (cost=9999), silent (no response)
- BFTConsensus: collects proposals, MAD-based outlier detection
- MAD Byzantine detection: $|c_k - \text{median}(c)| > \kappa \cdot \text{MAD}(c) \rightarrow \text{flag as Byzantine}$
- Weighted median: consensus cost from honest robot costs weighted by confidence
- Path selection: honest robot closest to consensus cost with highest confidence
- Multi-round experiment with history logging
- 5 tests passing including Byzantine detection and honest proposal validation

Data Flow

N robots → local A plans → proposals broadcast → MAD outlier detection → honest set H → weighted median cost → best path → execute*

④ Deep Technical Breakdown

Key Components

- SwarmRobot: local A* planner + fault mode implementation
- Proposal: (path, cost, confidence, signature) dataclass
- MAD detector: median absolute deviation for outlier detection
- Weighted median: sum-of-weights cutoff for robust central tendency
- Path selector: closest to c^* with highest confidence
- History logger: records per-round metrics

Pseudocode / Algorithm

```
BFT CONSENSUS ROUND:
    proposals = [robot_k.plan(grid, start, goal) for k in range(N)]
    valid = [p for p in proposals if p is not None]

    // Step 1: MAD outlier detection
    costs = [p.cost for p in valid]
    c_med = median(costs)
    MAD = median(abs(c - c_med) for c in costs)
    honest = [p for p in valid if abs(p.cost - c_med) <= kappa * MAD]
    n_byzantine = len(valid) - len(honest)

    // Step 2: Weighted median over honest set
    c_star = weighted_median([p.cost for p in honest], [p.confidence for p in honest])

    // Step 3: Select best path
    k_star = argmin over honest: |p.cost - c_star| - eta*p.confidence
    return ConsensusResult(path=honest[k_star].path, cost=c_star,
                           n_byzantine=n_byzantine)
```

Mathematical Formulation

```
BFT bound:  $n \geq 3f+1 \leftrightarrow f < n/3$  (classical result)
MAD:  $MAD = \text{median}(|c_k - \text{median}(c)|)$ 
Byzantine:  $B_k = 1[|c_k - \tilde{c}| > \kappa \cdot MAD]$ 
Weighted median:  $c^* = c_{\{j^*\}}$  where  $j^* = \min j: \sum_{i \leq j} w_{\{i\}} \geq (\sum w)/2$ 
Path selection:  $k^* = \text{argmin}_{k \in H} |c_k - c^*| - \eta \cdot \gamma_k$ 
```

Complexity & Assumptions

Time complexity: Single round: $O(N \cdot A^*)$ for proposals + $O(N \log N)$ for MAD + $O(N)$ for weighted median.

- $f < n/3$ Byzantine robots — MAD-based detection relies on honest majority
- Byzantine robots do not know the MAD scheme (non-adaptive attackers)
- Honest robots compute A* correctly on same map
- Proposal costs are real-valued and comparable across robots

⑤ Research Meaning

Research Question

Does weighted-median BFT consensus correctly identify and exclude Byzantine robots while selecting near-optimal navigation plans, and does it respect the $f < n/3$ tolerance bound?

Novelty

Applying BFT consensus specifically to multi-robot navigation plan selection, with weighted median aggregation incorporating robot confidence, is a novel framing. Most multi-robot work assumes cooperative agents.

Related Work

Lamport et al. (1982) Byzantine Generals. Castro & Liskov (1999) PBFT. Olfati-Saber (2006) swarm consensus. Vieira et al. — swarm consensus for navigation.

⑥ Results & Evidence

Byzantine detection correct in all rounds with $f=1$, $n=6$. Agreed cost consistent across rounds. Different fault types (random, constant_bad) both detected. 5 tests passing.

Metrics Used

- Byzantine detection rate
- Correct plan selection rate
- Agreed cost variance across rounds
- Comparison: majority vote vs. BFT weighted median

What Is Missing

No cryptographic authentication — sophisticated Byzantine agent can spoof honest behavior. No comparison with majority voting on same scenarios. No evaluation of κ threshold sensitivity.

⑦ Reviewer Critique

Weaknesses

- MAD detection not cryptographically secure: attacker knowing scheme can craft proposals within MAD threshold
- Statistical approximation of full PBFT — not a formal BFT protocol
- κ and η require tuning — no principled method
- Assumes all honest robots have the same map (violated if maps diverge)

Hidden Assumptions

- Attackers non-adaptive (do not know the MAD scheme)
- All honest robots have consistent maps
- Honest A* proposals vary within a reasonable range

Failure Scenarios

- Adaptive attacker: craft cost within MAD threshold but wrong path
- Many Byzantine robots ($f \geq n/3$): MAD detection fails
- Honest robots with very different maps: high honest cost variance resembles Byzantine

⑧ How to Turn This Into a Paper

Paper Title	Byzantine Fault-Tolerant Multi-Robot Navigation Consensus via Robust Statistical Aggregation
Venue	ICRA / AAMAS / ICDCS

Missing Experiments

- Comparison: BFT weighted median vs. naive mean vs. majority vote
- Adaptive attacker: can attacker craft proposals within MAD bounds?
- Scale: $f=1,2$ Byzantine out of $n=6,9$ robots
- Sensitivity: κ threshold effect on detection rate vs. false positive rate

Ablation Studies Needed

- Weighted median vs. uniform median vs. mean over honest set
- κ (MAD multiplier) sensitivity
- With vs. without confidence weighting in path selection
- Single round vs. multi-round consensus

9 Open Research Questions

100. Can cryptographic message authentication extend this to full PBFT?
101. Can Byzantine-robust aggregation from C26 be applied to federated learning (C16)?
102. How does communication latency affect the BFT consensus quality?
103. Can robots detect and blacklist confirmed Byzantine agents across rounds?

10 Two-Minute Explanation for Anyone

In a fleet of six robots, one might be hacked or broken and reporting wrong information. This module implements Byzantine fault-tolerant consensus: all robots propose their own navigation plans, then use a statistical test (median absolute deviation) to identify proposals that are statistical outliers — likely from faulty or malicious robots. The honest majority then computes a weighted median cost and selects the best path. The mathematical guarantee: as long as fewer than 1/3 of robots are faulty, the correct decision is always reached. Results: correct Byzantine detection in all simulated rounds.

Evaluation Scores

Dimension	Score	Comment
Originality	8/10	Genuinely novel
Technical Depth	8/10	Full maths + code
Publishability	8/10	Conference-ready now
PhD Potential	8/10	Strong chapter candidate

Part III — Final Synthesis

A. What Has Actually Been Built Scientifically

DynNav is a principled research framework for uncertainty-aware, risk-sensitive autonomous navigation. The 26 contributions form a coherent stack where uncertainty flows from sensing to planning, planning flows to safety, safety connects to security, and security feeds back to diagnosis. This is not 26 independent projects — it is an integrated system.

The scientific substance varies significantly across contributions. The strongest work (C18 formal safety shields, C14 causal attribution, C12 diffusion occupancy, C26 BFT consensus) represents genuine research contributions with mathematical depth and experimental validation. Several others (C21 PPO, C15 neuromorphic, C24 NeRF) are correctly designed frameworks awaiting training. The remainder are valuable systems integrations.

Fully validated contributions	C18 (STL+CBF shields): 93% violation reduction, <8% overhead
Strong partial implementations	C12, C14, C26, C03, C08, C16 — architecture correct, experiments limited
Framework prototypes (need training)	C21 (PPO), C13 (RSSM), C15 (SNN), C24 (NeRF) — no learning yet
Systems integration contributions	C02, C05, C09, C17, C19, C20 — valuable but not standalone research
Research concepts	C04, C06, C07, C10, C11, C22, C23, C25 — mix of integration and emerging ideas

B. The Hidden Strengths — Ideas the Author May Not Have Noticed

Three contributions stand out as potentially more novel than they appear at first glance:

- C14 + C20 combined (Causal Failure Pipeline): The integration of SCM do-calculus attribution with VLM scene description and STL violations into automated failure reports is unique in robotics. Most safety analysis is purely statistical. This causal + formal methods + language pipeline is a genuinely novel diagnostic architecture.
- C18 + C12 integration potential: The CBF safety filter currently uses a simple distance-to-obstacle barrier function. If the diffusion occupancy model (C12, once trained) provides probabilistic obstacle estimates, the CBF $h(x)$ could incorporate distributional uncertainty — creating a stochastic CBF with probabilistic forward invariance guarantees. This combination does not exist in the literature.
- C26 + C16 robustness angle: The weighted median aggregation in C26 (Byzantine consensus) is exactly the Byzantine-robust aggregation needed for federated learning (C16). Combining these creates Byzantine-robust federated navigation learning — a contribution that addresses both privacy and adversarial robustness simultaneously.

C. The Three Strongest Contributions for PhD Applications

- ★ C18 — Formal Safety Shields: Only contribution with complete implementation, principled mathematics, and quantitative experimental validation. Directly publishable at ICRA/RA-L with minor additions (formal proof, stochastic extension). This is the centerpiece of the research portfolio.
- ★ C14 — Causal Risk Attribution: Relatively novel in robotics. SCM + do-calculus applied to navigation failure attribution is not commonly seen. With learned structural equations, this becomes a strong publication. The combination with C20 creates a unique diagnostic pipeline.
- ★ C12 — Diffusion Occupancy Maps: Correct DDPM implementation, emerging research direction, clear upgrade path. Once a U-Net score network is trained on occupancy data, this becomes a conference-level publication. The CVaR-diffusion coupling is the novel contribution.

D. Evaluation as a Junior Researcher for a Robotics Lab

What would a senior PI see in this repository?

Strong signals

- Remarkable breadth: 26 modules spanning formal methods, generative AI, causal inference, neuromorphic computing, federated learning, and BFT — all connected by a coherent navigation research agenda
- Mathematical literacy: correct EKF/UKF, STL semantics, CBF conditions, DDPM reverse process, RSSM, SCM do-calculus, FedAvg DP — all implemented without textbook errors
- System design: the plug-in architecture (any risk source feeds the same CVaR planner) shows systems thinking beyond individual algorithm implementation
- Intellectual honesty: □ warnings on stub networks, experimental limitations documented, maturity levels acknowledged
- C18 result: 93% violation reduction with <8% overhead is a real, quantitative, reproducible result — the most important element of a research portfolio

Concerns

- Breadth vs. depth: 26 prototype-level contributions is weaker than 3 publication-level contributions. A PI will ask 'which 2-3 of these are you going to turn into papers?'
- Stub networks presented as working systems: if a candidate presents C21 (PPO) as 'I trained an RL navigation agent', a reviewer who asks about training curves will immediately see the stub. This must be framed carefully.
- No hardware comparison baseline: every result is simulation-only. Real robot validation of even 3 key results would significantly strengthen the portfolio.
- Publication record: a repository alone is not peer-reviewed work. One arXiv preprint (even a workshop paper) based on C18 would dramatically increase credibility.

Verdict

This repository demonstrates the breadth of knowledge, system-level thinking, and mathematical foundation of a strong PhD candidate. The depth of implementation varies significantly across contributions, but the strongest work (C18, C14, C12) shows genuine research potential. Recommended action: focus energy on publishing C18 as a standalone paper, training C12's score network, and connecting C14+C20 into a causal diagnosis pipeline paper. These three actions would transform a strong portfolio into an excellent one.

E. How to Present DynNav Without Overclaiming

Correct framing for different contexts:

For a CV	'Developed DynNav, a 26-module research framework for uncertainty-aware autonomous navigation, with contributions in STL/CBF formal safety (93% violation reduction), causal failure attribution via SCM, and DDPM-based probabilistic risk estimation.'
For a research statement	'DynNav investigates how formal safety verification, probabilistic uncertainty reasoning, and causal inference can coexist in a single autonomous navigation system. The strongest contribution — runtime STL+CBF safety shields — demonstrates 93% violation reduction with formal guarantees. This work motivates a PhD research agenda in formal methods for learning-augmented navigation.'
For an interview	'I built a modular navigation system with 26 research modules. The one I am most proud of is the formal safety shield — a runtime filter that mathematically guarantees no unsafe command is ever executed, with under 8% efficiency cost. I also implemented a causal model for failure attribution and a diffusion-based probabilistic occupancy predictor — both awaiting more experimental validation to become publishable contributions.'
What NOT to say	'I built an AI system with deep learning, reinforcement learning, diffusion models, NeRF, neuromorphic computing, and Byzantine fault tolerance.' — this sounds impressive but a reviewer will ask about each and find the stubs.

